DOCUMENTACION TECNICA

Tipiti Books

Plataforma de libros infantiles personalizados con IA

Documentos incluidos:

1. PDR - Product Definition Report v2.0
2. Tech Spec - Especificaciones Tecnicas v1.0
3. Image Generation Guidelines v2.0

Fecha: Febrero 2026
Estado: Aprobado / Borrador

Contenido

1. Product Definition Report

PDR: Tipiti Books

1. Problema de Negocio
2. Propuesta de Valor
3. Usuario Objetivo
4. Arquitectura de Datos
5. KPIs de Éxito
6. Modelo de Negocio
7. Alcance del MVP (Fase 1)
8. Consideraciones Especiales
9. Gaps Identificados y Recomendaciones
10. Próximos Pasos (Pipeline)

2. Technical Specifications

Tipiti Books — Technical Specifications

1. Resumen Ejecutivo
2. Stack Tecnológico
3. Arquitectura
4. Base de Datos
5. API Specifications
6. Autenticación y Seguridad
7. Integraciones Externas
8. Performance
9. Error Handling
10. Deployment
11. Testing Strategy
12. Consideraciones Futuras (Post-MVP)
13. Gotchas y Auto-Blindaje
14. Convenciones de Código

3. Image Generation Guidelines

TIPITI-IMAGE-GENERATION-GUIDELINES.md

1. Decisión Arquitectónica: Pre-Generación por Variantes
2. Stack de Generación de Imágenes
3. Modelo de Datos — Pre-Generación
4. Prompt Builder — Especificación Técnica
5. Pipeline de Pre-Generación — Flujo Técnico
6. Pipeline de Compra — Composición en Tiempo Real
7. Reglas de Prompting para Consistencia — Cheat Sheet
8. Quality Assurance — Validación de Variantes
9. Evolución por Fases
10. Formatos de Imagen y Resolución
11. Parámetros API de Referencia Rápida
12. Fuentes y Referencias Técnicas

01

Product Definition Report

PDR v2.0 - Modelo de pre-generacion por variantes

PDR: Tipiti Books

> *Product Definition Report Estado: APROBADO Fecha: 2026-02-18 Versión: 2.0 (actualizado con modelo de pre-generación por variantes) Documento técnico de referencia:* `TIPITI-IMAGE-GENERATION-GUIDELINES.md` *v2.0*

1. Problema de Negocio

El Dolor

Los libros infantiles personalizados de calidad — como los de Wonderbly — cuestan **$60+ USD** enviados a Chile y Latinoamérica. Para una familia chilena, esto equivale a más de $55.000 CLP solo por un libro. Las opciones locales o más económicas tienen ilustraciones genéricas, tipo "AI slop": personajes plásticos estilo Pixar, sin alma artística, que cualquier padre medianamente exigente descarta al verlas.

El mercado latinoamericano carece de una opción que combine personalización real del personaje, calidad artística de ilustración (estilo acuarela/pintado a mano), y un precio accesible. Los padres, abuelos y familiares quieren regalar algo verdaderamente especial y único — un libro donde el niño se reconozca — pero hoy deben elegir entre pagar $60+ USD por calidad internacional o conformarse con opciones locales mediocres.

El Costo

- **Costo monetario para el comprador:** $60+ USD por libro de Wonderbly enviado a Chile (precio + envío internacional + impuestos). Esto excluye a gran parte del mercado latinoamericano.
- **Costo de oportunidad:** Compradores potenciales abandonan la compra al ver el precio final o la baja calidad de las alternativas locales.
- **Competidores existentes:** Existen opciones en el mercado pero sus ilustraciones son feas, genéricas, o claramente generadas por IA sin curación. No hay un estándar de calidad premium accesible en la región.

Situación Actual

Hoy los compradores en LATAM tienen estas opciones:

Opción	Precio	Calidad	Personalización
Wonderbly / Hooray Heroes	$60+ USD a Chile	Alta	Limitada (configurador básico)
Opciones locales LATAM	$15-30 USD	Baja ("AI slop")	Variable
Libros artesanales (Etsy, locales)	$40-80 USD	Variable	Manual, lenta, sin escalabilidad
No comprar / regalo alternativo	$0	N/A	N/A

Ninguna opción combina: precio accesible + calidad artística premium + personalización real del personaje + experiencia de regalo premium.

---

2. Propuesta de Valor

En Una Frase

> *Una plataforma web chilena que genera libros infantiles personalizados con ilustraciones de alta calidad estilo acuarela, usando IA para consistencia de personajes, a un precio accesible (~$40 USD) con empaque premium de regalo.*

"Un Wonderbly para Latinoamérica, powered by AI."

Flujo Principal (Happy Path) — V1

```
1. EXPLORAR     ' Comprador entra a tipitibooks.com y elige un libro del catálogo
2. PERSONALIZAR ' Configura el personaje del niño/a:
- Nombre del niño/a
- Género (niño/niña)
- Pelo: 4 colores (rubio, castaño, pelirrojo, negro) × 2 tipos (liso, ondulado)
- Piel: 3 tonos (clara, mate, oscura)
- Lentes: sí/no
- Ojos: puntitos negros (NO customizables — decisión de diseño)
- Niño = pelo corto por defecto, Niña = pelo más largo
3. DEDICAR      ' Escribe una dedicatoria personal
4. PREVIEW      ' Sistema busca la variante pre-generada que coincide con la selección
 y muestra el libro con el nombre del niño/a superpuesto como texto
(las ilustraciones ya existen, pre-generadas y curadas por Sofi)
5. PAGAR        ' Checkout con MercadoPago (~$40 USD)
6. COMPONER     ' Sistema genera el archivo final para impresión:
- Capa 1: Ilustraciones pre-generadas (de la variante seleccionada)
- Capa 2: Texto narrativo con {name} reemplazado por el nombre del niño/a
- Capa 3: Dedicatoria en la página designada
7. IMPRIMIR     ' Envío a imprenta (proveedor TBD)
8. QA           ' Sofi recibe el libro impreso, revisa calidad
9. EMPACAR      ' Empaque premium de regalo (la experiencia de unboxing ES parte de la marca)
10. ENVIAR      ' Envío al comprador/destinatario final
```

Arquitectura de 3 Capas Independientes

Tipiti Books se estructura en **3 capas que se crean, escalan y versionan de forma totalmente independiente**. Esta separación es el principio arquitectónico central del producto:

```
CAPA 3 — TEXTO PERSONALIZADO (por compra)
Nombre del niño ({name}), dedicatoria, idioma
Se genera al instante. Costo: $0. Escala: infinita.
                            $
CAPA 2 — PERSONAJE (por variante, 96 combinaciones)
Ilustraciones pre-generadas con FLUX Kontext + curadas
Se genera UNA vez por libro. Costo: ~$42-120. Escala: fija.
```

```
                                $
  CAPA 1 — EL CUENTO (por libro del catálogo)
  Escenas, ambientes, easter eggs, estructura narrativa
  Se crea y cura UNA vez. Costo: tiempo de Sofi. Escala: manual.
```

¿Por qué importa? Cada capa tiene su propio ciclo de vida:

Capa	¿Quién la crea?	¿Cuándo se crea?	¿Cuánto cuesta por venta?	¿Cómo escala?
1. Cuento	Sofi (con IA para textos, manual para diseño de escenas)	Al agregar un libro nuevo al catálogo	$0	Sofi agrega nuevos libros al catálogo
2. Personaje	Pipeline de pre-generación (FLUX Kontext) + curación manual de Sofi	Una vez por libro (96 variantes × 11 páginas = 1,056 imágenes)	$0	Fija: 96 variantes cubren todas las combinaciones de V1
3. Texto	Sistema automático (reemplazo de `{name}` + dedicatoria)	En cada compra, al instante	$0	Infinita: solo es string replacement

Beneficios de la separación:

- **Agregar un idioma (V2)** = solo agregar una sub-capa de texto. No toca ilustraciones ni personajes.
- **Agregar un libro al catálogo** = crear Capa 1 (escenas) + generar Capa 2 (96 variantes). La Capa 3 ya funciona automáticamente.
- **Cambiar un texto narrativo** = solo editar `scenes.text_narrative`. Las ilustraciones no se tocan.
- **Mejorar una ilustración** = solo regenerar esa página en Capa 2. El cuento y los textos no se tocan.

Modelo de Pre-Generación por Variantes

Decisión arquitectónica clave (v2.0): La Capa 2 (Personaje) se implementa mediante pre-generación por variantes. Se pre-generan TODAS las 96 combinaciones posibles de personaje para cada libro del catálogo, y Sofi las revisa y aprueba una por una. Al momento de la compra, el sistema simplemente busca la variante que coincide con la selección del comprador y superpone la Capa 3 (Texto).

Separación imagen/texto: Las ilustraciones (Capa 2) no contienen texto ni nombres. El nombre del niño/a es una variable `{name}` en la Capa 3 que se superpone al componer el archivo final. Esto permite que las mismas ilustraciones sirvan para cualquier nombre.

Atributo	Opciones	Valores
Género	2	niño, niña
Color de pelo	4	rubio, castaño, pelirrojo, negro
Tipo de pelo	2	liso, ondulado
Tono de piel	3	claro, mate, oscuro
Lentes	2	sí, no

Ojos	1 (fijo)	puntitos negros
Total combinaciones	**96**	por libro

Beneficios del modelo:

- **$0 USD en generación por venta** (costo fijo ~$42-120 USD por libro, una sola vez)
- **Calidad garantizada:** Sofi revisa y aprueba cada variante antes de publicar
- **Experiencia instantánea:** El preview se muestra al instante (no hay espera de generación)
- **Consistencia perfecta:** Todas las páginas de una variante fueron generadas y aprobadas juntas

Especificaciones de Impresión

- **Tamaño de página:** 22 × 18 cm
- **Total de páginas:** 20 (1 portada + 9 spreads dobles horizontales + 1 página final)
- **Regla de composición:** Personaje NUNCA en el centro de la imagen (zona del lomo/pliegue del libro). Debe quedar centrado a la izquierda o derecha, alternando entre páginas.
- **Espacio negativo para texto:** Posición dinámica alternando por página (arriba-izquierda → abajo-izquierda → arriba-derecha → abajo-derecha) para dar dinamismo visual.
- **Espacio reservado para segundo idioma:** Desde V1, el diseño de cada spread reserva espacio para un bloque de texto adicional (para habilitar bilingüe en V2 sin rediseñar).

Contenido Narrativo

- **Generación:** Textos generados por IA, optimizados para la edad objetivo del libro.
- **Edición:** Sofi tiene panel de administración para editar tanto los textos narrativos como las descripciones visuales de cada escena. Esto permite correcciones de hablantes nativos (ej: un padre alemán sugiere una mejor frase) y ajustes artísticos por escena.
- **Personalización por nombre:** Los textos narrativos usan la variable `{name}` que se reemplaza con el nombre del niño/a al momento de la compra. El nombre NUNCA aparece dentro de las ilustraciones.
- **Estilo de texto para "Buenas Noches" (0-3 años):** Frases muy cortas, vocabulario simple, abundantes onomatopeyas ("Shhh...", "Zzzzz", "Muuu", "¡Splash!").

Estilo de Ilustración

- Acuarela/pintado a mano, con pinceladas visibles, paleta pastel cálida.
- **NO planas:** Ricas en detalles que los niños aman buscar — animalitos escondidos, peluches recurrentes, objetos curiosos. Ejemplos: un osito dormido en la caja de juguetes, un ratoncito asomándose por la ventana, una mariposa posada en una flor diferente en cada escena.
- Estos elementos recurrentes ("easter eggs") deben mantener consistencia visual a lo largo de todo el libro usando FLUX Kontext multi-referencia.
- Ojos del personaje principal como **puntitos negros** (estilo ilustración infantil clásica, evitando deliberadamente el look 3D/Pixar).
- **Regla de exclusión de texto en imágenes:** Las ilustraciones NO deben contener letras, palabras ni texto de ningún tipo. Todo texto es superpuesto posteriormente.

Flujos V2 (Fase 2+)

- **Segundo personaje** (hermano/a, amigo/a) — Nota: genera 96×96=9,216 combinaciones si se pre-genera. Se recomienda modelo híbrido: personaje 1 pre-generado, personaje 2 generado on-demand.
- **Foto del regalador** integrada en la dedicatoria/última página
- **Título y color de portada personalizables**
- **Peluche editable** que acompaña al personaje en el libro
- **Opción bilingüe:** Texto en segundo idioma (alemán, inglés, francés, etc.) aprovechando el espacio negativo reservado desde V1
- **Bundle regalo:** Libro + peluche físico que coincide con el personaje del libro (alto margen, alto valor percibido)

Flujos Alternativos y Edge Cases

- **Variante no aprobada aún:** Si una combinación específica no ha sido aprobada todavía, mostrar estado "próximamente" y ofrecer combinaciones similares disponibles.
- **Comprador quiere ver preview antes de imprimir:** Opción "regalar sin spoiler" — comprador recibe link de preview para verificar, pero el destinatario no ve nada hasta recibir el paquete.
- **Corrección post-compra:** Si el comprador detecta un error en los datos (nombre, dedicatoria), puede solicitar corrección antes de confirmar impresión.
- **Problema con impresión:** Sofi hace QA manual de cada libro; si hay defecto, se reimprime antes de enviar.

---

3. Usuario Objetivo

Persona Principal: El Comprador de Regalos

- **Rol:** Padres, abuelos, tíos, amigos — cualquier persona que quiera regalar un libro personalizado a un niño/a
- **Contexto:** Buscan un regalo significativo y único para ocasiones especiales
- **Ocasiones de compra:** Nacimientos, baby showers, cumpleaños, Navidad, Día del Niño, bautizos
- **Nivel técnico:** Variable (desde tech-savvy hasta abuelos con smartphone). La experiencia debe ser intuitiva para todos.
- **Dispositivo principal: Mobile-first** (la mayoría comprará desde el celular)
- **Frecuencia de uso:** Eventual (1-4 veces al año, por ocasiones)
- **Perfil demográfico:** Familias tanto bilingües como monolingües. El primer libro ("Buenas Noches") apunta a padres con hijos de 0-3 años.

Persona Secundaria: Sofi (Operadora/Admin)

- **Rol:** Dueña y operadora de Tipiti Books
- **Interacción:** Panel de administración para:
  - **Pre-generación de variantes:** Lanzar el pipeline de generación de las 96 variantes por libro
  - **Curación de variantes:** Revisar cada variante generada, aprobar/rechazar/regenerar páginas individuales
  - Editar textos narrativos y descripciones de escenas (prompts)

  - Gestionar el catálogo de libros y el estado de cada variante
  - Gestionar pedidos y fulfillment
- **Dispositivo:** Desktop (panel de admin)

TAM Estimado

- Chile: ~250.000 nacimientos/año + cumpleaños de niños 0-6 años
- Mercado inicial: Familias chilenas de NSE ABC1-C2 + comunidad de expatriados/bilingües
- Expansión LATAM: México, Colombia, Argentina (Fase 3+)
- Mercado comparable: Wonderbly genera >$100M USD anuales a nivel global

---

4. Arquitectura de Datos

Input — Qué entra al sistema

Dato	Tipo	Fuente	Obligatorio
Nombre del niño/a	Texto (form)	Comprador	Sí
Género	Selección (niño/niña)	Comprador	Sí
Color de pelo	Selección (4 opciones)	Comprador	Sí
Tipo de pelo	Selección (liso/ondulado)	Comprador	Sí
Tono de piel	Selección (3 opciones)	Comprador	Sí
¿Usa lentes?	Toggle (sí/no)	Comprador	Sí
Dedicatoria	Texto libre	Comprador	Sí (V1)
Idioma del libro	Selección	Comprador	Sí (V1: 1 idioma)
Datos de pago	MercadoPago	Comprador	Sí
Dirección de envío	Form	Comprador	Sí
Foto del regalador	Imagen	Comprador	No (V2)
Segundo idioma	Selección	Comprador	No (V2)

Output — Qué sale del sistema

Dato	Tipo	Destino	Formato
Preview del libro	Ilustraciones + texto superpuesto	Comprador (pantalla)	JPEG/WebP (preview)
Archivo para impresión	Ilustraciones + texto compuesto	Imprenta	PDF high-res (300dpi, CMYK)

Libro personalizado impreso	Libro físico 22×18cm, 20 páginas	Cliente final	Impresión offset/digital
Confirmación de pedido	Email	Comprador	Email transaccional
Notificación de envío	Email	Comprador	Email con tracking
Factura/boleta	Documento fiscal	Comprador	PDF o electrónica (SII Chile)

Entidades Principales (Modelo Conceptual v2.0)

Entidad	Descripción	Relaciones
Book	Plantilla de un libro del catálogo (escenas, estilo, textos base)	Tiene muchas Scenes. Tiene muchas CharacterVariants.
Scene	Una escena/spread individual del libro (descripción visual, texto narrativo con `{name}`, posición de cámara, iluminación)	Pertenece a un Book.
CharacterVariant	Una combinación específica de atributos de personaje (género + pelo + piel + lentes) para un Book. Contiene el status de aprobación.	Pertenece a un Book. Tiene muchas VariantPages.
VariantPage	Una imagen pre-generada para una escena específica dentro de una variante específica. Incluye image_url, prompt_used, quality_score.	Pertenece a un CharacterVariant y a una Scene.
Order	Pedido de un comprador (nombre del niño + variante + dedicatoria + pago + envío)	Referencia un CharacterVariant, un Book.
User	Comprador (guest o registrado)	Tiene muchos Orders.

> *Para schemas SQL detallados y pipeline de pre-generación, ver* `TIPITI-IMAGE-GENERATION-GUIDELINES.md` *§3, §5 y §6.*

---

5. KPIs de Éxito

Métrica Principal

50-70 libros vendidos en el primer mes tras el lanzamiento.

Métricas del Funnel

Etapa	Métrica	Target
Visitas	Usuarios únicos en tipitibooks.com	Tracking

Intención	Usuarios que inician personalización	>30% de visitas
Configuración completa	Usuarios que terminan de personalizar	>50% de los que inician
Preview	Usuarios que ven el preview del libro	>70% de configuración completa
Compra	Conversión a pago	>15% de los que ven preview
Satisfacción	NPS post-entrega	>8/10

Métricas Operativas (Nuevas v2.0)

Métrica	Target
Variantes aprobadas por libro	96/96 (100% cobertura)
Tiempo de aprobación por variante	<5 min revisión manual
Tiempo total de pre-generación por libro	<24 horas (batch completo)
Uptime del preview instantáneo	>99.5%

Hitos Temporales

- **Semana 2:** Primer libro con todas las 96 variantes pre-generadas y aprobadas
- **Semana 3:** Primer libro impreso y entregado (proof of concept end-to-end)
- **Mes 1:** 50-70 unidades vendidas
- **Mes 3:** Segundo título en catálogo

Marketing

- Canal principal: **Influencers mamás** (Instagram, TikTok)
- La experiencia de unboxing del empaque premium ES el contenido de marketing
- Sin Google Ads en la fase inicial
- Estrategia pre-lanzamiento: Waitlist / landing page "Reserva el tuyo"

6. Modelo de Negocio

Monetización

Venta directa de libros personalizados (e-commerce B2C). Cada libro se vende individualmente. No hay modelo de suscripción en V1.

Pricing (actualizado v2.0)

Componente	Costo estimado	Nota

Pre-generación de variantes (fal.ai)	~$42-120 USD/libro	**Costo ÚNICO por libro del catálogo**
Composición texto + imagen (por venta)	~$0 USD	Proceso automático sin costo de IA
Impresión (proveedor TBD)	~$8-12 USD/libro	Por unidad
Envío (proveedor → Sofi)	~$3-5 USD	Por unidad
Empaque premium	~$3-5 USD	Por unidad
Envío (Sofi → cliente)	~$5-8 USD (Chile)	Por unidad
Costo variable por venta	**~$19-30 USD**	Sin costo de generación de IA
Precio de venta	**~$40 USD (~$37.000 CLP)**	—
Margen bruto	**~33-55%**	Mejor que v1.0 por eliminación de costo IA por venta

Amortización del costo de pre-generación: El costo fijo de ~$42-120 USD por libro se amortiza rápidamente. Con 50 ventas del primer libro, el costo por unidad baja a <$1-2.40 USD.

Competencia

Competidor	Qué hacen	Precio a Chile	Nuestra diferencia
Wonderbly	Libros personalizados premium, ilustración profesional	$60+ USD	Precio 35% menor, enfocado en LATAM, empaque de regalo
Hooray Heroes	Personalización por foto, estilo cartoon	$50+ USD	Calidad artística superior (acuarela vs cartoon genérico)
Opciones locales LATAM	Libros con IA genérica	$15-30 USD	Calidad de ilustración premium, consistencia de personaje, no "AI slop"
Libros artesanales (Etsy)	Hechos a mano	$40-80 USD	Escalable, entrega más rápida, precio competitivo

Diferenciadores Clave

- **Calidad artística:** Estilo acuarela/pintado a mano con detalles ricos, no "AI slop"
- **Consistencia de personaje:** FLUX Kontext + curación manual garantizan que el niño se ve igual en todas las páginas
- **Precio accesible para LATAM:** ~$40 USD vs $60+ de la competencia
- **Experiencia de regalo premium:** Empaque de regalo incluido, no es "solo un libro"
- **Detalles que enamoran:** Animalitos escondidos, peluches recurrentes, easter eggs que los niños buscan en cada lectura
- **Preview instantáneo:** Gracias a la pre-generación, el comprador ve su libro personalizado al instante (sin esperas de generación)

7. Alcance del MVP (Fase 1)

Features Core (Debe tener)

- **Catálogo web + Configurador de personaje**
  - Landing page con catálogo (1-2 libros iniciales, empezando por "Buenas Noches")
  - Configurador visual del personaje: nombre, género, pelo (4 colores × liso/ondulado), piel (3 tonos), lentes (sí/no)
  - Ojos como puntitos negros (no customizables)
  - Campo de dedicatoria
  - Preview instantáneo del libro personalizado (lookup de variante pre-generada + texto superpuesto)
  - Mobile-first responsive
- **Sistema de pre-generación y curación de variantes (Admin)**
  - Pipeline batch para generar las 96 variantes de un libro: portrait base → character sheet → 11 páginas por variante
  - Panel de curación: Sofi revisa cada variante, aprueba/rechaza/regenera páginas individuales
  - Dashboard de progreso: cuántas variantes aprobadas vs pendientes vs rechazadas por libro
  - Motor: FLUX Kontext Pro vía fal.ai
  - Ver `TIPITI-IMAGE-GENERATION-GUIDELINES.md` para especificaciones completas
- **Composición de archivo final y checkout**
  - Motor de composición: superponer texto narrativo (con `{name}` reemplazado) + dedicatoria sobre las ilustraciones pre-generadas
  - Generación de PDF high-res para impresión (300dpi, CMYK)
  - Pago con MercadoPago
  - Flujo de pedido: pago → composición → envío a imprenta (proveedor TBD) → envío a Sofi → QA + empaque premium → envío al cliente
  - Email de confirmación de pedido
  - Email de notificación de envío
- **Panel de administración (Sofi)**
  - Gestión del catálogo de libros y sus escenas
  - Editar textos narrativos de cada escena (correcciones de idioma, ajustes de estilo)
  - Editar descripciones visuales de escenas (ajustar prompts de generación)
  - Lanzar y monitorear pipeline de pre-generación
  - Curación de variantes (aprobar/rechazar/regenerar)
  - Gestionar pedidos y estados de fulfillment

Features Diferidas (Fase 2+)

- Segundo personaje (hermano/a, amigo/a) — modelo híbrido: personaje 1 pre-generado, personaje 2 on-demand
- Foto del regalador en la dedicatoria/última página

- Título y color de portada personalizables
- Peluche editable que acompaña al personaje
- Opción bilingüe (segundo idioma en espacio negativo reservado)
- Bundle libro + peluche físico
- Style LoRA entrenado para estilo Tipiti propio
- QA semi-automatizado (CLIP scoring para pre-filtrar variantes antes de revisión manual)
- Notificaciones por WhatsApp
- Segundo y tercer título en catálogo
- Cuenta de usuario con historial de pedidos

Features Diferidas (Fase 3+)

- Expansión a otros países LATAM (México, Colombia, Argentina)
- Múltiples estilos artísticos por libro (gouache, lápiz, etc.)
- Self-hosted inference (GPU pool) para reducir costos a escala
- App móvil nativa
- Gift cards / códigos de regalo
- Programa de referidos

Explícitamente Fuera de Alcance (MVP)

- **No PDF/digital:** Solo libro físico impreso. No se ofrece versión digital descargable.
- **No generación on-demand:** Las imágenes se pre-generan por variantes. No hay generación de imágenes al momento de la compra.
- **No auto-agendar citas ni dashboard complejo:** La operación la maneja Sofi manualmente con apoyo del panel admin.
- **No Google Ads ni paid acquisition:** Marketing solo vía influencers orgánico en V1.
- **No multi-idioma en la UI:** La interfaz web es en español (Chile). El libro puede estar en otro idioma pero la plataforma no.
- **No creación de cuenta obligatoria:** Guest checkout con email. Cuenta opcional post-compra.

8. Consideraciones Especiales

Requisitos No Funcionales

- **Autenticación:** Guest checkout (email) para compradores. Login con email/password para Sofi (admin). No se requiere OAuth ni SSO en V1.
- **Roles/Permisos:** 2 roles — Comprador (guest) y Admin (Sofi). En V1, Sofi es la única admin.
- **Pagos/Billing:** MercadoPago (Chile). Soporte para tarjetas de crédito/débito y transferencia bancaria.
- **Datos Sensibles:** Datos de pago manejados 100% por MercadoPago (PCI compliance delegada). No se almacenan datos de tarjeta. Datos personales del niño (nombre) requieren política de privacidad clara.

- **Integraciones externas:**
  - fal.ai API (pre-generación de imágenes — solo usado en admin, no en flujo de compra)
  - MercadoPago API (pagos)
  - Proveedor de impresión API (TBD — placeholder)
  - Servicio de email transaccional (Resend)
- **Multi-idioma (contenido):** Los textos del libro pueden estar en cualquier idioma. La UI de la plataforma es solo en español (V1).
- **Multi-tenant:** No. Single-tenant (Tipiti Books es el único operador).
- **Offline:** No requerido.
- **Performance:** El preview es instantáneo (lookup de variante pre-generada). La composición del archivo final para impresión es automática y rápida (<30 segundos). La pre-generación de variantes es asíncrona (solo la usa Sofi desde el admin, no afecta al comprador).

Restricciones Conocidas

- **Proveedor de impresión TBD:** Aún no se define el proveedor final. El sistema debe ser agnóstico al proveedor, con una interfaz/adaptador que permita cambiar de proveedor sin rediseñar el flujo.
- **Fulfillment manual:** En V1, Sofi recibe cada libro, hace QA y empaca manualmente. Esto limita la capacidad a ~20-30 pedidos por semana (estimación).
- **Empresa en trámite:** La sociedad/inicio de actividades en SII está en proceso. Facturación electrónica se implementará cuando esté listo.
- **Dominio en proceso:** tipitibooks.com (o similar) está en proceso de adquisición.
- **Token limit de FLUX Kontext:** 512 tokens máximo por prompt. Los prompts deben ser optimizados para no exceder este límite.
- **Formato de impresión fijo:** Páginas de 22×18 cm. El personaje no puede quedar en el centro (zona del pliegue). Esto impone restricciones de composición que deben estar codificadas en el prompt builder.
- **96 variantes por libro:** Cada nuevo libro del catálogo requiere generación y curación de 1,056 imágenes (96 variantes × 11 páginas). Esto representa un esfuerzo inicial significativo pero amortizable.

Riesgos Identificados

Riesgo	Impacto	Mitigación
Curación de 96 variantes es muy lenta	Medio — retrasa lanzamiento de nuevos libros	Priorizar variantes más comunes primero (ej: pelo castaño/negro). Implementar QA semi-automatizado con CLIP scoring en Fase 2.
Inconsistencia del personaje entre páginas de una variante	Alto — producto defectuoso	Pipeline de QA por variante: Sofi revisa las 11 páginas juntas. Regeneración selectiva de páginas individuales. FLUX Kontext tiene 94% fidelidad.
Costos de pre-generación mayores a lo estimado	Bajo — costo es fijo y predecible	~$42-120 USD por libro es controlable. Monitorear costo por imagen. Migración a LoRA en Fase 2-3.

Proveedor de impresión no cumple calidad	Alto — producto defectuoso, marca dañada	QA manual por Sofi antes de cada envío. Tener 2-3 proveedores evaluados como backup.
Cuello de botella en fulfillment manual	Medio — limita crecimiento	Limitar ventas iniciales. Plan de transición a fulfillment service en Fase 2.
fal.ai downtime durante pre-generación	Bajo — no afecta ventas, solo la creación de nuevos libros	Pre-generación se hace con anticipación. Inngest maneja reintentos automáticos. Endpoint de fallback (Replicate).
Derechos de imagen / privacidad de menores	Alto — riesgo legal	Política de privacidad clara. Solo se almacena el nombre del niño (no fotos en V1). Consentimiento explícito de los padres.
Textos generados por IA inapropiados o con errores	Medio — experiencia del usuario	Panel de edición para Sofi. Revisión humana de todos los textos antes de publicar un libro en catálogo.
Combinatoria explota en V2 con segundo personaje	Medio — limita escalabilidad	Modelo híbrido: personaje 1 pre-generado (96), personaje 2 on-demand. Evaluar selective pre-generation para combos más populares.

9. Gaps Identificados y Recomendaciones

> *Observaciones que surgieron durante la entrevista de negocio*

- **Autenticación de comprador:** No se discutió en detalle. **Recomendación:** Guest checkout con email (mínima fricción) + opción de crear cuenta post-compra para tracking de pedido. Sin OAuth en V1.
- **Notificaciones del pedido:** No se definió el flujo de comunicación post-compra. **Recomendación:** Email de confirmación → email cuando se envía → email con tracking. WhatsApp como canal en Fase 2.
- **Tiempos de entrega:** No se definieron tiempos estimados. **Recomendación:** Estimar y comunicar claramente en la página de producto. Ejemplo: "Tu libro estará listo en 5-8 días hábiles." (Nota: con pre-generación, se elimina el tiempo de generación de imágenes. El tiempo es: composición instantánea + impresión 3-5 días + envío a Sofi 1-2 días + QA/empaque 1 día + envío final 2-3 días Chile).
- **Política de devoluciones/reimpresión:** No se definió. **Recomendación:** Si el QA detecta defecto, Sofi reimprime sin costo. Si el cliente reporta problema post-entrega, evaluar caso por caso (reimpresión o reembolso parcial).
- **Facturación Chile (SII):** La sociedad está en trámite. **Recomendación:** Implementar integración con facturación electrónica (boleta/factura) cuando esté lista la empresa. En MVP, emitir boletas manuales si es necesario.
- **Límite de capacidad operacional:** Con fulfillment manual, Sofi puede procesar ~20-30 pedidos/semana. **Recomendación:** Comunicar "edición limitada" o "producción artesanal" como parte del branding mientras se escala. Implementar cola de pedidos si la demanda supera la capacidad.
-

Contenido de libros — pipeline de creación: Los textos los genera IA pero Sofi los edita. **Recomendación:** Crear un flujo de "publicación de libro" donde: IA genera borrador → Sofi edita/ajusta → hablantes nativos revisan (para otros idiomas) → se publica al catálogo. Cada libro es un "template" curado, no generación on-the-fly.

- **Priorización de variantes:** Con 96 variantes por libro, ¿se lanzan todas al mismo tiempo? **Recomendación:** Priorizar las 20-30 combinaciones más comunes (pelo castaño/negro, piel clara/mate, sin lentes) para lanzamiento rápido, y completar las restantes en las semanas siguientes.

---

10. Próximos Pasos (Pipeline)

Una vez aprobado este PDR, los siguientes skills del pipeline generarán:

- **Tech Spec** — Definición de stack técnico, arquitectura, esquema de base de datos, integraciones
- **User Stories** — Historias de usuario completas con criterios de aceptación (INVEST)
- **Wireframes** — Wireframes low-res desktop + mobile para cada pantalla
- **Stitch Prompts** — Design System + prompts para convertir wireframes a UI real
- **Master Blueprint** — Plan de implementación por fases con estimaciones

Documentos de Referencia

- `TIPITI-IMAGE-GENERATION-GUIDELINES.md` v2.0 — Fuente de verdad técnica para generación de imágenes (pre-generación por variantes, schemas SQL, prompt builder, API specs, pipeline de curación)
- `Tipiti_Books_Investigacion_Consistencia_Personajes.md` — Investigación inicial sobre modelos y técnicas
- `Comparativa_Enfoques_Consistencia.md` — Análisis comparativo IP-Adapter vs FLUX Kontext

PDR v2.0 generado con el pipeline de App Factory Actualizado para reflejar el modelo de pre-generación por variantes

02

Technical Specifications

Tech Spec v1.0 - Stack, arquitectura, base de datos y APIs

Tipiti Books — Technical Specifications

Tech Spec v1.0 Estado: BORRADOR Fecha: 2026-02-18 PDR de referencia: PDR-tipiti-books.md v2.0 Documento de imágenes: TIPITI-IMAGE-GENERATION-GUIDELINES.md v2.0

1. Resumen Ejecutivo

Problema (del PDR)

Los libros infantiles personalizados de calidad cuestan $60+ USD enviados a Chile/LATAM, y las opciones locales tienen ilustraciones genéricas tipo "AI slop". No hay una opción que combine personalización real, calidad artística premium y precio accesible.

Solución Técnica

Plataforma e-commerce con configurador de personaje que busca variantes pre-generadas (96 combinaciones por libro), superpone textos personalizados (nombre + dedicatoria) y genera un PDF para impresión. Las ilustraciones se pre-generan via FLUX Kontext Pro y se curan manualmente antes de publicar al catálogo.

Complejidad Estimada

Medio — E-commerce B2C con integraciones externas (pagos, generación de imágenes, email), panel de admin con pipeline de pre-generación batch, y motor de composición de archivos para impresión. No requiere tiempo real, multi-tenancy, ni alta disponibilidad en V1.

2. Stack Tecnológico

2.1 Tabla Resumen

Capa	Tecnología	Versión	Justificación
Framework	Next.js	15	App Router + Server Components para catálogo SSR. Server Actions para mutaciones. RSC reduce JS al cliente en páginas de catálogo.
Language	TypeScript	5.x	Strict mode. Contratos tipados entre frontend, backend y Supabase.

Styling	Tailwind CSS	4.x	Utility-first, ideal para mobile-first responsive. Rápido de iterar.
Components	shadcn/ui	latest	Componentes accesibles, customizables, no-dependency. Perfecto para admin + storefront.
State Mgmt	Zustand	5.x	Estado del configurador de personaje (selección + preview) necesita estado global entre componentes. Ligero.
Animations	Framer Motion	12.x	Transiciones del configurador de personaje, page flip preview, micro-interactions de la landing.
Validation	Zod	3.x	Schemas compartidos client/server. Validación de formularios y API inputs.
Backend	Supabase	latest	Auth + PostgreSQL + Storage + Edge Functions. Todo-en-uno reduce complejidad operacional para MVP de 1 persona.
Database	PostgreSQL (via Supabase)	15+	Relacional, RLS nativo, JSONB para datos semi-estructurados.
Auth	Supabase Auth	latest	Email/password para admin. Guest checkout no requiere auth del comprador.
Storage	Supabase Storage	latest	3 buckets: illustrations, illustrations-hires, portraits. CDN integrado para servir previews.
Background Jobs	Inngest	latest	Orquestación multi-step del pipeline de pre-generación (96 variantes × 11 páginas). Reintentos, concurrency control, observabilidad.
Payments	MercadoPago	latest	Checkout Pro (redirect). Proveedor dominante en Chile. Tarjetas + transferencia.
Email	Resend	latest	Emails transaccionales (confirmación, envío). API simple, React Email templates.
AI/Images	fal.ai (FLUX Kontext Pro)	latest	Pre-generación de ilustraciones. Solo usado en admin, no en flujo de compra.
Image Processing	Sharp	latest	Composición de texto sobre ilustraciones. Generación de PDF para impresión.

PDF Generation	@react-pdf/renderer o pdf-lib	latest	Composición del archivo final para impresión (ilustraciones + texto + dedicatoria).
Upscaling	Real-ESRGAN via fal.ai	latest	Upscale de ilustraciones a resolución de impresión (300 DPI).
Analytics	PostHog	latest	Funnel de conversión, eventos custom. Self-hosteable si crece.
Monitoring	Sentry	latest	Error tracking frontend + serverless functions. Alertas en Slack.
Hosting	Vercel	-	Deploy automático desde GitHub. Edge network. Integración nativa con Next.js.
Testing	Vitest + Playwright	latest	Unit tests (Vitest) + E2E del flujo de compra (Playwright). Coverage mínimo en MVP.

2.2 Decisiones Técnicas Importantes

Inngest sobre Supabase Edge Functions para background jobs

- Razón: El pipeline de pre-generación requiere orquestación multi-step (portrait → character sheet → 11 páginas × 96 variantes = 1,056+ pasos). Inngest maneja reintentos, concurrency, timeouts y observabilidad out-of-the-box.
- Trade-off: Dependencia adicional. Capa gratuita de Inngest es generosa (25K events/mes).
- Reevaluar si: Solo necesitamos jobs simples sin orquestación compleja.

Zustand sobre URL state / Context para el configurador

- Razón: El configurador tiene ~6 campos interdependientes que afectan el preview en tiempo real. Zustand permite un store limpio con derivaciones (lookup de variante) sin prop-drilling.
- Trade-off: Dependencia adicional (3KB gzipped).
- Reevaluar si: El configurador se simplifica a <3 campos.

MercadoPago Checkout Pro (redirect) sobre API de pagos directo

- Razón: Checkout Pro maneja toda la UI de pago, PCI compliance, y métodos locales (tarjetas, transferencia, cuotas). Mínimo esfuerzo de integración para V1.
- Trade-off: El usuario sale del sitio momentáneamente. Menor control sobre UX del checkout.
- Reevaluar si: La tasa de abandono en checkout es >30% (migrar a Checkout Bricks o API directo).

Sharp + pdf-lib sobre Puppeteer/Chrome para composición de PDF

- Razón: Composición directa de imágenes + texto es más rápida y ligera que renderizar HTML a PDF. Sharp maneja la superposición de texto, pdf-lib ensambla las páginas.
- Trade-off: Menos flexibilidad en layout que HTML-to-PDF.
- Reevaluar si: El diseño de página requiere layouts muy complejos con CSS.

Pre-generación por variantes sobre generación on-demand

- Razón: $0 de costo de IA por venta (vs $1-2), preview instantáneo, calidad curada y garantizada, no depende de fal.ai uptime para ventas.
- Trade-off: Costo fijo inicial (~$42-120 por libro), 1,056 imágenes que almacenar y curar.
- Reevaluar si: V2 introduce segundo personaje (96^2 = 9,216 combinaciones). Modelo híbrido recomendado.

2.3 Lo Que NO Se Incluye (y por qué)

Tecnología	Razón de exclusión	Agregar en
Redis/Upstash	No se necesita rate limiting ni caché agresivo con <100 usuarios iniciales	Fase 2 si hay abuso
tRPC	Server Actions de Next.js cubren las necesidades de API. No hay consumidor externo.	Fase 3 si se necesita API pública
WebSockets / Realtime	No hay funcionalidad que requiera tiempo real para el comprador	Fase 2 si se agrega tracking live
WhatsApp API	Canal de notificación secundario	Fase 2
Stripe	MercadoPago es el estándar en Chile. Stripe no tiene igual penetración local.	Fase 3 expansión internacional
Docker	Vercel maneja el deploy. No se necesita containerización en V1.	Fase 3 si se migra a VPS
LoRA Training	El estilo se logra con prompts optimizados en V1	Fase 2 para estilo Tipiti propio

3. Arquitectura

3.0 Principio Fundacional: Arquitectura de 3 Capas Independientes

Toda la arquitectura técnica de Tipiti Books se deriva de un principio central: el producto se compone de **3 capas que se crean, escalan y versionan de forma totalmente independiente**. Esta separación dicta las decisiones de base de datos, almacenamiento, pipeline de generación y flujo de compra.

```
CAPA 3 — TEXTO PERSONALIZADO (varía por compra)

Nombre del niño ({name}), dedicatoria, idioma.
Se compone al instante por string replacement. Costo por venta: $0.
Escala: infinita. No toca imágenes.
DB: orders.child_name + orders.dedication + scenes.text_narrative
Runtime: composición server-side (Sharp + pdf-lib)
                                $
CAPA 2 — PERSONAJE (varía por variante, 96 combinaciones fijas)

Ilustraciones pre-generadas con FLUX Kontext Pro, curadas por Sofi.
```

```
Se genera UNA vez por libro del catálogo. Costo fijo: ~$42-120 USD.
Escala: fija (96 combinaciones en V1).
DB: character_variants + variant_pages
Storage: buckets illustrations + illustrations-hires + portraits
                                $
CAPA 1 — EL CUENTO (varía por libro del catálogo)

Escenas, ambientes, composición visual, easter eggs, narrativa.
Se crea y cura UNA vez por título. Escala: Sofi agrega nuevos libros.
DB: books + scenes
Es el "template" inmutable que alimenta las otras dos capas.
```

Implicaciones técnicas de la separación:

Operación	Capas afectadas	Capas NO afectadas
Agregar un libro al catálogo	Capa 1 (crear escenas) + Capa 2 (pre-generar 96 variantes)	Capa 3 (funciona automáticamente)
Agregar un idioma (V2)	Capa 3 (agregar campo en `text_narrative` JSONB)	Capa 1 y 2 (ilustraciones no cambian)
Corregir un texto narrativo	Capa 1 (editar `scenes.text_narrative`)	Capa 2 y 3 (ilustraciones y composición no cambian)
Mejorar una ilustración	Capa 2 (regenerar `variant_pages` específica)	Capa 1 y 3 (cuento y textos no cambian)
Nueva venta	Capa 3 (componer texto + imagen)	Capa 1 y 2 (ya existen, solo se leen)
Segundo personaje (V2)	Capa 2 (modelo híbrido: pre-gen + on-demand)	Capa 1 (escenas se adaptan) + Capa 3 (segundo {name})

Esta separación se refleja directamente en el schema de DB (§4), la estructura de Storage (§4.3), el pipeline de Inngest (§3.4), y las API specifications (§5).

3.1 Diagrama de Alto Nivel

```
 Comprador                    Next.js 15                     Supabase
 (Mobile)         Vercel (App Router)       PostgreSQL
               À                          À Auth
                     Storage
  Store-     Admin
  front      Panel

    ,

        <

¼                ¼                  ¼

 MercadoPago      Inngest            Resend
  (Pagos)        (Jobs batch        (Email)
     pre-gen)
  ,

  ¼
```

```
    fal.ai
   FLUX Kontext  ilustraciones    ¶
   + Real-ESRGAN    (Storage)
```

3.2 Arquitectura de Carpetas

```
tipiti-books/
 src/
    app/                          # Next.js App Router
       (storefront)/             # Grupo: páginas públicas
          page.tsx              # Landing / catálogo
          libro/[slug]/
             page.tsx          # Detalle del libro
             personalizar/
                 page.tsx      # Configurador de personaje
          preview/[orderId]/
             page.tsx          # Preview del libro personalizado
          checkout/
             page.tsx          # Resumen + redirect a MercadoPago
             success/page.tsx  # Post-pago exitoso
             failure/page.tsx  # Post-pago fallido
          layout.tsx            # Layout storefront (nav, footer)

       admin/                    # Grupo: panel de administración
          layout.tsx            # Layout admin (sidebar, auth check)
          page.tsx              # Dashboard overview
          libros/
             page.tsx          # Lista de libros del catálogo
             [bookId]/
                 page.tsx      # Detalle del libro + escenas
                 variantes/
                    page.tsx  # Grid de 96 variantes + estados
                 variante/[variantId]/
                     page.tsx  # Detalle: 11 páginas, aprobar/rechazar
          pedidos/
             page.tsx          # Lista de pedidos
             [orderId]/
                 page.tsx      # Detalle del pedido + fulfillment
          generacion/
              page.tsx          # Monitor de jobs de pre-generación

       api/                      # API Routes
          webhooks/
             mercadopago/route.ts  # Webhook de pago
             inngest/route.ts      # Inngest event handler
          compose/route.ts      # Endpoint de composición PDF

       layout.tsx                # Root layout
       globals.css               # Tailwind + custom styles

    components/
       ui/                       # shadcn/ui components
       storefront/               # Componentes del storefront
          character-configurator.tsx
          book-preview.tsx
          book-card.tsx
          checkout-summary.tsx
       admin/                    # Componentes del admin
           variant-grid.tsx
           variant-detail.tsx
```

```
            scene-editor.tsx
            order-table.tsx

      lib/
         supabase/
            client.ts             # Supabase browser client
            server.ts             # Supabase server client (SSR)
            admin.ts              # Supabase service role client
            types.ts              # Generated types from Supabase
         inngest/
            client.ts             # Inngest client
            functions/
                generate-all-variants.ts
                generate-single-variant.ts
                upscale-variant-pages.ts
         fal/
            client.ts             # fal.ai client wrapper
            prompt-builder.ts     # buildCharacterBlock, buildFullPrompt
         compose/
            text-overlay.ts       # Superposición de texto sobre imagen
            pdf-builder.ts        # Ensamblaje de PDF para impresión
         mercadopago/
            client.ts             # MercadoPago SDK wrapper
         utils/
             variant-lookup.ts     # findVariant()
             constants.ts          # BUYER_TO_VARIANT_MAP, etc.

      stores/
         configurator-store.ts     # Zustand store para el configurador

      schemas/                      # Zod schemas
         order.ts
         book.ts
         variant.ts

      emails/                       # React Email templates
          order-confirmation.tsx
          shipping-notification.tsx

   supabase/
      migrations/                   # SQL migrations
         001_books_and_scenes.sql
         002_character_variants.sql
         003_orders.sql
         004_rls_policies.sql
      seed.sql                      # Seed data (libro "Buenas Noches" + escenas)

   public/
      ...                           # Static assets

   .env.local                        # Environment variables (local)
   next.config.ts
   tailwind.config.ts
   tsconfig.json
   package.json
   inngest.config.ts                 # Inngest configuration
```

3.3 Componentes del Sistema

Storefront (Next.js App Router — RSC)

- Propósito: Catálogo de libros, configurador de personaje, preview, checkout

- Tecnología: Server Components para catálogo (SSR), Client Components para configurador (interactivo)
- Se comunica con: Supabase (lectura de libros, variantes), MercadoPago (checkout), Zustand (estado del configurador)
- Escala: Vercel Edge Network sirve estáticos. PostgreSQL connection pooling via Supabase.

Admin Panel (Next.js — protegido por auth)

- Propósito: Gestión de catálogo, pre-generación de variantes, curación, fulfillment
- Tecnología: Server Components + Server Actions para mutaciones
- Se comunica con: Supabase (CRUD), Inngest (lanzar jobs), fal.ai (indirectamente via Inngest)
- Escala: Single user (Sofi). No requiere optimización.

Pipeline de Pre-Generación (Inngest)

- Propósito: Generar las 96 variantes × 11 páginas para un libro
- Tecnología: Inngest functions con steps, concurrency limit, reintentos
- Se comunica con: fal.ai (generación de imágenes), Supabase Storage (almacenar resultados), Supabase DB (actualizar estados)
- Escala: Concurrency controlada por Inngest. ~4-8 generaciones simultáneas.

Motor de Composición (Server-side)

- Propósito: Superponer texto personalizado sobre ilustraciones pre-generadas y generar PDF para impresión
- Tecnología: Sharp (image processing) + pdf-lib (PDF assembly)
- Se comunica con: Supabase Storage (leer ilustraciones), Supabase DB (leer textos narrativos)
- Escala: Composición es ligera (<30 segundos por libro). Se ejecuta on-demand al confirmar pedido.

3.4 Flujo de Datos

```
=== FLUJO DE COMPRA (tiempo real) ===

Comprador selecciona atributos

 ¼
Zustand store actualiza estado

 ¼
findVariant(bookId, selection) ' Supabase query

 ¼
variant_pages[] (imágenes pre-generadas) + scenes[] (textos con {name})

 ¼
Preview: imágenes + texto con nombre superpuesto (client-side)

 ¼
Checkout ' MercadoPago redirect

 ¼
Webhook MercadoPago ' crear Order ' status: 'paid'

 ¼
Composición PDF (server-side):
- Descargar ilustraciones hi-res de Storage
- Reemplazar {name} en textos narrativos
```

```
- Superponer texto sobre imágenes (Sharp)
- Ensamblar PDF 20 páginas (pdf-lib)
- Subir PDF a Storage

 ¼
Order status: 'ready_to_print' ' Email a Sofi

 ¼
Sofi envía a imprenta ' QA ' Empaque ' Envío


=== FLUJO DE PRE-GENERACIÓN (admin, async) ===

Sofi clickea "Generar variantes" en admin

 ¼
Inngest event: 'book/generate-all-variants'

 ¼
Para cada combinación (96 total):
  Step 1: Generar portrait (fal.ai text-to-image)
  Step 2: Generar character sheet (fal.ai image-to-image)
  Step 3: Generar 11 páginas (fal.ai image-to-image × 11)
  Step 4: Upscale a hi-res (Real-ESRGAN × 11)
  Step 5: Actualizar estado en DB ' 'generated'

 ¼
Sofi revisa en admin ' Aprobar / Rechazar / Regenerar

 ¼
Cuando 96/96 aprobadas ' Libro se publica al catálogo
```

4. Base de Datos

4.1 Modelo de Datos (Diagrama ER)

```
books (1) < (many) scenes
books (1) < (many) character_variants
character_variants (1) < (many) variant_pages
variant_pages (many) > (1) scenes
orders (many) > (1) character_variants
orders (many) > (1) books
```

4.2 Schema Completo

Los schemas SQL completos están definidos en `TIPITI-IMAGE-GENERATION-GUIDELINES.md` §3 (secciones 3.1 a 3.5). Aquí se documenta el schema con RLS policies y detalles adicionales.

```
-- ============================================
-- Tabla: books
-- Propósito: Catálogo de libros (templates)
-- ============================================
CREATE TABLE books (
```

```
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug TEXT UNIQUE NOT NULL,
  title_template TEXT NOT NULL,           -- 'Buenas Noches, {name}'
  description TEXT,
  style_prompt TEXT NOT NULL,             -- STYLE_BLOCK fijo
  style_lora_url TEXT,
  total_scenes INTEGER NOT NULL DEFAULT 11,
  target_age TEXT DEFAULT '0-3',
  default_language TEXT DEFAULT 'es',
  available_languages TEXT[] DEFAULT '{"es"}',

-- Estado del catálogo
  total_variants INTEGER DEFAULT 96,
  approved_variants INTEGER DEFAULT 0,
  is_published BOOLEAN DEFAULT FALSE,

-- Impresión
  page_width_cm DECIMAL(4,1) DEFAULT 22.0,
  page_height_cm DECIMAL(4,1) DEFAULT 18.0,
  total_pages INTEGER DEFAULT 20,

  cover_template_url TEXT,
  price_clp INTEGER NOT NULL,             -- precio en CLP
  price_usd DECIMAL(10,2),               -- precio en USD (referencia)

  created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
  updated_at TIMESTAMPTZ DEFAULT now() NOT NULL
);

-- RLS: Libros publicados son públicos; todos los libros visibles para admin
ALTER TABLE books ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Libros publicados visibles para todos"
  ON books FOR SELECT
  USING (is_published = true);

CREATE POLICY "Admin ve todos los libros"
  ON books FOR ALL
  USING (auth.jwt() ->> 'role' = 'admin');

COMMENT ON TABLE books IS 'Catálogo de libros personalizables. Cada libro es un template con escenas y variantes
pre-generadas.';

-- ============================================
-- Tabla: scenes
-- Propósito: Escenas/spreads de un libro
-- ============================================
CREATE TABLE scenes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  book_id UUID REFERENCES books(id) ON DELETE CASCADE NOT NULL,
  scene_number INTEGER NOT NULL,          -- 0=portada, 1-9=spreads, 10=final

  visual_description TEXT NOT NULL,
  camera_angle TEXT DEFAULT 'medium_shot',
  lighting TEXT DEFAULT 'warm',
  emotion TEXT DEFAULT 'peaceful',
  suggested_seed INTEGER,

  recurring_elements TEXT[],              -- ['sleeping_mouse', 'teddy_bear', 'butterfly']

  character_position TEXT DEFAULT 'left', -- 'left', 'right' (NUNCA 'center')
  text_position TEXT DEFAULT 'top_left',
  text_position_secondary TEXT,
```

```
 text_narrative JSONB NOT NULL,         -- {"es": "Buenas noches, {name}...", "de": "Gute Nacht, {name}..."}

 created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
 updated_at TIMESTAMPTZ DEFAULT now() NOT NULL,

 UNIQUE(book_id, scene_number)
);

ALTER TABLE scenes ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Escenas de libros publicados son públicas"
 ON scenes FOR SELECT
 USING (EXISTS (SELECT 1 FROM books WHERE books.id = scenes.book_id AND books.is_published = true));

CREATE POLICY "Admin CRUD escenas"
 ON scenes FOR ALL
 USING (auth.jwt() ->> 'role' = 'admin');

-- ============================================
-- Tabla: character_variants
-- Propósito: Combinaciones pre-generadas de personaje
-- ============================================
CREATE TABLE character_variants (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 book_id UUID REFERENCES books(id) ON DELETE CASCADE NOT NULL,

 gender TEXT NOT NULL,                  -- 'boy', 'girl'
 hair_color TEXT NOT NULL,              -- 'blonde', 'brown', 'red', 'black'
 hair_type TEXT NOT NULL,               -- 'straight', 'wavy'
 skin_tone TEXT NOT NULL,               -- 'light', 'medium', 'dark'
 has_glasses BOOLEAN NOT NULL DEFAULT FALSE,
 eye_style TEXT DEFAULT 'black_dots',

 portrait_url TEXT,
 character_sheet_urls JSONB,

 status TEXT DEFAULT 'pending',         -- 'pending', 'generating', 'generated', 'approved', 'rejected'
 approved_by TEXT,
 approved_at TIMESTAMPTZ,

 created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
 updated_at TIMESTAMPTZ DEFAULT now() NOT NULL,

 UNIQUE(book_id, gender, hair_color, hair_type, skin_tone, has_glasses)
);

-- Índice para lookup rápido en tiempo de compra
CREATE INDEX idx_variant_lookup
 ON character_variants(book_id, gender, hair_color, hair_type, skin_tone, has_glasses)
 WHERE status = 'approved';

ALTER TABLE character_variants ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Variantes aprobadas de libros publicados son públicas"
 ON character_variants FOR SELECT
 USING (
 status = 'approved' AND
 EXISTS (SELECT 1 FROM books WHERE books.id = character_variants.book_id AND books.is_published = true)
);

CREATE POLICY "Admin CRUD variantes"
 ON character_variants FOR ALL
 USING (auth.jwt() ->> 'role' = 'admin');
```

```
-- ============================================
-- Tabla: variant_pages
-- Propósito: Páginas pre-generadas por variante
-- ============================================
CREATE TABLE variant_pages (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 variant_id UUID REFERENCES character_variants(id) ON DELETE CASCADE NOT NULL,
 scene_id UUID REFERENCES scenes(id) ON DELETE CASCADE NOT NULL,

 image_url TEXT NOT NULL,
 image_url_hires TEXT,

 prompt_used TEXT NOT NULL,
 seed_used INTEGER,
 generation_model TEXT NOT NULL,
 generation_cost DECIMAL(6,4),

 status TEXT DEFAULT 'generated',        -- 'generating', 'generated', 'approved', 'rejected'
 retry_count INTEGER DEFAULT 0,

 created_at TIMESTAMPTZ DEFAULT now() NOT NULL,

 UNIQUE(variant_id, scene_id)
);

ALTER TABLE variant_pages ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Páginas de variantes aprobadas son públicas"
 ON variant_pages FOR SELECT
 USING (
 EXISTS (
 SELECT 1 FROM character_variants cv
 WHERE cv.id = variant_pages.variant_id
 AND cv.status = 'approved'
 AND EXISTS (SELECT 1 FROM books WHERE books.id = cv.book_id AND books.is_published = true)
)
);

CREATE POLICY "Admin CRUD páginas"
 ON variant_pages FOR ALL
 USING (auth.jwt() ->> 'role' = 'admin');

-- ============================================
-- Tabla: orders
-- Propósito: Pedidos de compradores
-- ============================================
CREATE TABLE orders (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 variant_id UUID REFERENCES character_variants(id) NOT NULL,
 book_id UUID REFERENCES books(id) NOT NULL,

 child_name TEXT NOT NULL,
 dedication TEXT,
 language TEXT DEFAULT 'es',
 second_language TEXT,

 buyer_email TEXT NOT NULL,
 buyer_name TEXT NOT NULL,
 buyer_phone TEXT,

 shipping_address JSONB NOT NULL,
 shipping_city TEXT,
 shipping_region TEXT,
 shipping_country TEXT DEFAULT 'CL',
```

```
  payment_provider TEXT DEFAULT 'mercadopago',
  payment_id TEXT,
  payment_status TEXT DEFAULT 'pending',
  amount_paid DECIMAL(10,2),
  currency TEXT DEFAULT 'CLP',

  print_file_url TEXT,
  print_status TEXT DEFAULT 'pending',
  shipping_status TEXT DEFAULT 'pending',
  tracking_number TEXT,

  status TEXT DEFAULT 'created',

  created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
  updated_at TIMESTAMPTZ DEFAULT now() NOT NULL
);

CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_orders_buyer_email ON orders(buyer_email);

ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Admin gestiona pedidos"
 ON orders FOR ALL
 USING (auth.jwt() ->> 'role' = 'admin');

-- Trigger para updated_at
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
 NEW.updated_at = now();
 RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER books_updated_at BEFORE UPDATE ON books FOR EACH ROW EXECUTE FUNCTION update_updated_at();
CREATE TRIGGER scenes_updated_at BEFORE UPDATE ON scenes FOR EACH ROW EXECUTE FUNCTION update_updated_at();
CREATE TRIGGER character_variants_updated_at BEFORE UPDATE ON character_variants FOR EACH ROW EXECUTE FUNCTION
update_updated_at();
CREATE TRIGGER orders_updated_at BEFORE UPDATE ON orders FOR EACH ROW EXECUTE FUNCTION update_updated_at();
```

4.3 Storage / Buckets

Bucket	Contenido	Acceso	Límite
`portraits`	Portrait base de cada variante (1 por variante)	Privado (solo admin)	5MB por archivo
`illustrations`	Ilustraciones estándar (web preview, ~1200px)	Público (CDN)	5MB por archivo
`illustrations-hires`	Ilustraciones hi-res para impresión (300 DPI, ~4000px)	Privado (solo composición server-side)	20MB por archivo
`print-files`	PDFs finales para impresión	Privado (solo admin)	100MB por archivo

Estimación de almacenamiento por libro: 96 variantes × 11 páginas × 2 resoluciones ≈ 2,112 archivos. ~5GB por libro (incluyendo hi-res).

4.4 Migrations Strategy

Migraciones secuenciales en `supabase/migrations/`. Cada migración es un archivo SQL numerado. Se ejecutan con `supabase db push` (desarrollo) y `supabase db migrate` (producción).

```
supabase migration new create_books_and_scenes
supabase migration new create_character_variants
supabase migration new create_orders
supabase migration new create_rls_policies
```

5. API Specifications

5.1 Estilo de API

Server Actions para todas las mutaciones (Next.js 15 built-in). **Direct Supabase queries** desde Server Components para reads. **API Routes** solo para webhooks externos (MercadoPago, Inngest).

5.2 Server Actions

```
// ==============================
// Lookup de variante (storefront)
// ==============================

/**
* Busca la variante pre-generada que coincide con la selección del comprador.
* Se llama desde el configurador cuando el usuario completa la selección.
*
* Auth: Pública
*/
interface FindVariantRequest {
 bookId: string;           // UUID del libro
 gender: 'Niño' | 'Niña';
 hairColor: 'Rubio' | 'Castaño' | 'Pelirrojo' | 'Negro';
 hairType: 'Liso' | 'Ondulado';
 skinTone: 'Clara' | 'Mate' | 'Oscura';
 glasses: 'Sí' | 'No';
}

interface FindVariantResponse {
 variant: {
 id: string;
 pages: {
 sceneNumber: number;
 imageUrl: string;       // URL de ilustración (preview)
 textNarrative: string;  // Texto con {name} aún sin reemplazar
 textPosition: string;
}[];
} | null;
 available: boolean;         // false si la variante no está aprobada aún
}

// ==============================
// Crear pedido (checkout)
```

```
// ==============================

/**
* Crea un pedido y genera la preferencia de pago en MercadoPago.
*
* Auth: Pública (guest checkout)
*/
interface CreateOrderRequest {
 bookId: string;
 variantId: string;
 childName: string;          // 1-50 caracteres, sin caracteres especiales peligrosos
 dedication: string;         // 1-500 caracteres
 language: string;           // 'es' en V1
 buyerEmail: string;
 buyerName: string;
 buyerPhone?: string;
 shippingAddress: {
 street: string;
 number: string;
 apartment?: string;
 city: string;
 region: string;
 postalCode?: string;
 country: string;          // 'CL' en V1
};
}

interface CreateOrderResponse {
 orderId: string;
 mercadopagoUrl: string;     // URL de redirect a Checkout Pro
}

// ==============================
// Admin: Lanzar pre-generación
// ==============================

/**
* Dispara el pipeline de pre-generación de todas las variantes de un libro.
*
* Auth: Admin requerido
*/
interface GenerateAllVariantsRequest {
 bookId: string;
 concurrency?: number;       // Default: 4 (variantes simultáneas)
}

interface GenerateAllVariantsResponse {
 jobId: string;              // Inngest event ID
 totalVariants: number;      // 96
 estimatedMinutes: number;   // ~180-360 min
}

// ==============================
// Admin: Aprobar/Rechazar variante
// ==============================

/**
* Cambia el estado de una variante a 'approved' o 'rejected'.
*
* Auth: Admin requerido
*/
interface UpdateVariantStatusRequest {
 variantId: string;
 status: 'approved' | 'rejected';
```

```
}

// ==============================
// Admin: Regenerar página
// ==============================

/**
* Regenera una página específica de una variante.
*
* Auth: Admin requerido
*/
interface RegeneratePageRequest {
 pageId: string;             // variant_page.id
 newSeed?: number;           // Opcional: forzar un seed diferente
}
```

5.3 API Routes (Webhooks)

```
// POST /api/webhooks/mercadopago
// Webhook de notificación de pago de MercadoPago
// Verifica firma HMAC, actualiza order.payment_status,
// dispara composición de PDF si status = 'approved'

// POST /api/webhooks/inngest
// Handler de Inngest para functions registradas
// Inngest SDK maneja routing automáticamente

// POST /api/compose
// Endpoint interno para composición de PDF
// Llamado después de pago exitoso
// Auth: Internal (API key o Inngest)
```

5.4 Validación

Zod schemas compartidos entre client y server. Ubicados en `src/schemas/`.

```
// src/schemas/order.ts
import { z } from 'zod';

export const createOrderSchema = z.object({
 bookId: z.string().uuid(),
 variantId: z.string().uuid(),
 childName: z.string().min(1).max(50).regex(/^[\p{L}\s'-]+$/u, 'Nombre inválido'),
 dedication: z.string().min(1).max(500),
 language: z.enum(['es']),   // Expandir en V2
 buyerEmail: z.string().email(),
 buyerName: z.string().min(1).max(100),
 buyerPhone: z.string().optional(),
 shippingAddress: z.object({
 street: z.string().min(1),
 number: z.string().min(1),
 apartment: z.string().optional(),
 city: z.string().min(1),
 region: z.string().min(1),
 postalCode: z.string().optional(),
 country: z.literal('CL'),  // Solo Chile en V1
}),
});
```

6. Autenticación y Seguridad

6.1 Flujo de Auth

```
Comprador:
 No requiere autenticación. Guest checkout con email.
 El email se usa para confirmación y tracking.

Admin (Sofi):
 Email + Password via Supabase Auth
 ' Login en /admin/login
 ' JWT con claim custom { role: 'admin' }
 ' Middleware de Next.js verifica JWT en /admin/*
 ' RLS en Supabase usa auth.jwt() ->> 'role' = 'admin'
```

6.2 Roles y Permisos

Rol	Puede hacer	No puede hacer
Guest (Comprador)	Ver catálogo, configurar personaje, ver preview, pagar, ver estado de su pedido por link	Acceder a /admin, ver otros pedidos, modificar datos del catálogo
Admin (Sofi)	Todo lo de guest + CRUD libros/escenas, pre-generar variantes, aprobar/rechazar, gestionar pedidos	N/A (full access en V1)

6.3 Protección de Rutas

Ruta Pattern	Acceso	Redirect si no auth
/	Público	N/A
/libro/*	Público	N/A
/checkout/*	Público	N/A
/preview/*	Público (por link con orderId)	N/A
/admin/*	Admin autenticado	/admin/login
/api/webhooks/*	Verificación de firma	401

6.4 Security Checklist

- [ ] Passwords hasheados (Supabase Auth usa bcrypt)
- [ ] JWT con expiración (24h para admin, configurable)

- [ ] HTTPS enforced (Vercel default)
- [ ] CORS configurado (solo dominio propio)
- [ ] Rate limiting en checkout (Vercel built-in, 100 req/min)
- [ ] SQL injection prevenido (Supabase client usa queries parametrizadas)
- [ ] XSS protection (React escapes by default, CSP headers)
- [ ] CSRF — Server Actions de Next.js incluyen protección CSRF
- [ ] Secrets en env vars (nunca hardcoded)
- [ ] File uploads: no hay uploads de usuario en V1 (V2: foto del regalador)
- [ ] RLS habilitado en todas las tablas
- [ ] Webhook de MercadoPago verificado con HMAC signature
- [ ] childName sanitizado (regex + max length) para prevenir XSS en composición

---

7. Integraciones Externas

7.1 fal.ai (Generación de Imágenes)

- **Propósito**: Pre-generación de ilustraciones (pipeline admin)
- **API Docs**: https://fal.ai/docs
- **Auth method**: API Key (`FAL_KEY`)
- **Endpoints que usamos**:
  - `fal-ai/flux-pro/kontext/text-to-image` — portrait base
  - `fal-ai/flux-pro/kontext` — image-to-image (páginas con referencia)
  - `fal-ai/flux-pro/kontext/max/multi` — multi-referencia para consistency
  - `fal-ai/real-esrgan` — upscaling a hi-res
- **Costo estimado**: ~$42-120 por libro (1,056 imágenes + upscale)
- **Gotchas conocidos**: Rate limits en capa gratuita; respuestas pueden tardar 10-30s por imagen; URLs de resultado expiran (descargar y almacenar en Supabase Storage inmediatamente)
- **Fallback si falla**: Replicate como provider alternativo para FLUX Kontext. Inngest maneja reintentos automáticos (3 intentos con backoff exponencial).

7.2 MercadoPago (Pagos)

- **Propósito**: Procesamiento de pagos para Chile
- **API Docs**: https://www.mercadopago.cl/developers/es/docs
- **Auth method**: Access Token (`MERCADOPAGO_ACCESS_TOKEN`)
- **Endpoints que usamos**:
  - Checkout Pro (crear preferencia de pago → redirect)
  - Webhook de notificación (IPN)
- **Costo estimado**: Comisión ~3.49% + IVA por transacción
-

Gotchas conocidos: Webhook puede enviar notificaciones duplicadas (idempotencia requerida en handler). Checkout Pro redirect puede tener latencia. Status "in_process" para tarjetas requiere polling.

- **Fallback si falla**: No hay fallback en V1. Mostrar mensaje de error y pedir reintentar.

7.3 Resend (Email Transaccional)

- **Propósito**: Emails de confirmación de pedido y notificación de envío
- **API Docs**: https://resend.com/docs
- **Auth method**: API Key (`RESEND_API_KEY`)
- **Endpoints que usamos**: Envío de emails con React Email templates
- **Costo estimado**: Gratis hasta 3,000 emails/mes (suficiente para V1)
- **Gotchas conocidos**: Dominio verificado requerido para evitar spam. Rate limit de 10 emails/segundo.
- **Fallback si falla**: Reintentar 3 veces. Si persiste, loguear error y notificar a admin.

7.4 Inngest (Background Jobs)

- **Propósito**: Orquestación del pipeline de pre-generación de variantes
- **API Docs**: https://www.inngest.com/docs
- **Auth method**: Signing key (`INNGEST_SIGNING_KEY`) + Event key (`INNGEST_EVENT_KEY`)
- **Funciones registradas**:
  - `book/generate-all-variants` — orquesta la generación de 96 variantes
  - `variant/generate-single` — genera 1 variante (portrait + sheet + 11 páginas)
  - `variant/upscale-pages` — upscale de las 11 páginas a hi-res
- **Costo estimado**: Gratis hasta 25,000 events/mes
- **Gotchas conocidos**: Functions tienen timeout de 5 min por step. Cada step debe ser idempotente. Usar `step.run()` para cada operación que puede fallar.
- **Fallback si falla**: Inngest maneja reintentos automáticamente. Admin puede relanzar variantes fallidas manualmente.

7.5 PostHog (Analytics)

- **Propósito**: Tracking del funnel de conversión
- **Auth method**: Project API Key (público, client-side)
- **Eventos trackeados**: `page_view`, `configurator_started`, `configurator_completed`, `preview_viewed`, `checkout_started`, `payment_completed`
- **Costo estimado**: Gratis hasta 1M events/mes
- **Gotchas**: Respetar Do Not Track. Cookie-less mode para GDPR.

7.6 Sentry (Error Monitoring)

- **Propósito**: Error tracking en frontend y serverless functions
- **Auth method**: DSN (`SENTRY_DSN`)
- **Costo estimado**: Gratis hasta 5,000 errors/mes
- **Integración**: `@sentry/nextjs` con source maps en Vercel

8. Performance

8.1 Targets

Métrica	Target	Máximo Aceptable
First Contentful Paint (landing)	<1.5s	2.5s
Time to Interactive (configurador)	<2.5s	4s
Preview load (variante lookup + render)	<1s	2s
Composición PDF (server-side)	<30s	60s
API Response (checkout creation)	<2s	5s

8.2 Estrategias de Optimización

- **Image optimization**: Next.js Image component con formato WebP/AVIF para previews. Lazy loading para imágenes debajo del fold.
- **Caching**: Variantes aprobadas se cachean en Vercel Edge (ISR o fetch cache). Supabase CDN para imágenes en `illustrations` bucket.
- **Code splitting**: App Router hace splitting automático por ruta. Configurador como Client Component se carga con dynamic import.
- **DB indexing**: Índice compuesto en `character_variants` para lookup instantáneo (ver §4.2). Índice en `orders(status)` para panel admin.
- **Prefetching**: Prefetch de las ilustraciones de la variante más probable mientras el usuario configura.

8.3 Escalabilidad

10x (500-700 ventas/mes): Sin cambios técnicos. Vercel escala automáticamente. Supabase Pro plan para más connections. Storage ~50GB.

100x (5,000-7,000 ventas/mes): Agregar Redis para caché de variantes. Considerar CDN dedicado (Cloudflare R2). Migrar composición de PDF a worker dedicado.

9. Error Handling

9.1 Error Codes

```
enum AppErrorCode {
// Client (400s)
 VALIDATION_ERROR = "VALIDATION_ERROR",
```

```
  VARIANT_NOT_FOUND = "VARIANT_NOT_FOUND",       // Variante no aprobada o no existe
  VARIANT_NOT_AVAILABLE = "VARIANT_NOT_AVAILABLE", // Variante existe pero no está aprobada
  ORDER_NOT_FOUND = "ORDER_NOT_FOUND",
  PAYMENT_FAILED = "PAYMENT_FAILED",
  UNAUTHORIZED = "UNAUTHORIZED",
  FORBIDDEN = "FORBIDDEN",

 // Server (500s)
  INTERNAL_ERROR = "INTERNAL_ERROR",
  GENERATION_FAILED = "GENERATION_FAILED",        // fal.ai error
  COMPOSITION_FAILED = "COMPOSITION_FAILED",      // PDF composition error
  PAYMENT_WEBHOOK_ERROR = "PAYMENT_WEBHOOK_ERROR",
  STORAGE_ERROR = "STORAGE_ERROR",                // Supabase Storage error
 }
```

9.2 Logging Strategy

- **Sentry**: Todos los errores de servidor (500s) y errores no capturados en cliente
- **Console (Vercel logs)**: Info-level para operaciones importantes (pedido creado, pago recibido, composición completada)
- **Inngest Dashboard**: Estado de todos los jobs de pre-generación (éxitos, fallos, reintentos)
- **PostHog**: Eventos de negocio (no errores técnicos)

9.3 User-Facing Errors

- **Variante no disponible**: Toast con "Esta combinación aún no está disponible. Prueba con otra opción." + sugerir variantes cercanas disponibles
- **Error de pago**: Página de error con botón de reintentar + email de soporte
- **Error de composición**: No visible al usuario. Se loguea internamente y Sofi recibe alerta.
- **Error genérico**: Toast con "Algo salió mal. Por favor intenta de nuevo."

---

10. Deployment

10.1 Environments

Env	URL	Propósito	Deploy trigger
Development	localhost:3000	Dev local	Manual
Preview	*.vercel.app	PR preview	Push a PR branch
Production	tipitibooks.com	Live	Push a `main`

10.2 Environment Variables

Públicas (client-safe):

```
NEXT_PUBLIC_SUPABASE_URL          # URL del proyecto Supabase
NEXT_PUBLIC_SUPABASE_ANON_KEY     # Clave anon de Supabase (segura para client)
NEXT_PUBLIC_POSTHOG_KEY           # Project API key de PostHog
NEXT_PUBLIC_POSTHOG_HOST          # Host de PostHog
NEXT_PUBLIC_SENTRY_DSN            # DSN de Sentry (client)
NEXT_PUBLIC_MERCADOPAGO_PUBLIC_KEY # Clave pública de MercadoPago
```

Secretas (server-only):

```
SUPABASE_SERVICE_ROLE_KEY         # Clave service role (bypasses RLS)
FAL_KEY                           # API key de fal.ai
MERCADOPAGO_ACCESS_TOKEN          # Token de acceso de MercadoPago
MERCADOPAGO_WEBHOOK_SECRET        # Secreto para verificar webhooks
RESEND_API_KEY                    # API key de Resend
INNGEST_SIGNING_KEY               # Signing key de Inngest
INNGEST_EVENT_KEY                 # Event key de Inngest
SENTRY_AUTH_TOKEN                 # Token para source maps upload
```

10.3 CI/CD

```
Push to PR:
 ' Vercel Preview Deploy (automático)
 ' TypeScript typecheck (tsc --noEmit)
 ' ESLint

Push to main:
 ' Vercel Production Deploy (automático)
 ' supabase db push (migrations)
```

No hay pipeline de testing automatizado en V1 (MVP). Se agrega en Fase 2.

10.4 Infrastructure

- **Hosting**: Vercel (Hobby plan para empezar, Pro cuando sea necesario)
- **CDN**: Vercel Edge Network (global) + Supabase Storage CDN
- **DNS**: Configurar en registrar del dominio → apuntar a Vercel
- **SSL**: Automático vía Vercel (Let's Encrypt)

11. Testing Strategy

11.1 Approach

Tipo	Herramienta	Coverage Target	Qué se testea
Unit	Vitest	Funciones críticas	prompt-builder, variant-lookup, text-overlay, Zod schemas

E2E	Playwright	Flujos happy path	Configurador → Preview → Checkout flow

11.2 Testing Commands

```
# Unit tests
pnpm test              # Vitest run
pnpm test:watch        # Vitest watch mode

# E2E tests
pnpm test:e2e          # Playwright
pnpm test:e2e:ui       # Playwright UI mode
```

11.3 E2E Flows Críticos

- **Happy path completo**: Seleccionar libro → Configurar personaje → Ver preview → Completar datos → Redirect a MercadoPago
- **Variante no disponible**: Seleccionar combinación no aprobada → Mostrar mensaje + sugerencias
- **Admin: aprobar variante**: Login admin → Navegar a libro → Revisar variante → Aprobar

12. Consideraciones Futuras (Post-MVP)

Feature/Mejora	Impacto Técnico	Fase Estimada
Segundo personaje	Modelo híbrido: pre-gen personaje 1 + on-demand personaje 2. Agrega tabla `second_characters`. Inngest function nueva.	Fase 2
Opción bilingüe	Agregar `second_language` a orders. Composición PDF renderiza 2 bloques de texto por página.	Fase 2
Style LoRA	Entrenar LoRA con fal-ai/flux-kontext-trainer. Agregar `style_lora_url` a books (ya en schema).	Fase 2
QA semi-automatizado	CLIP scoring en pipeline de pre-generación. Auto-reject si score < threshold.	Fase 2
WhatsApp notifications	Integrar Twilio/YCloud API. Agregar `notification_channel` a orders.	Fase 2
Gift cards	Nueva tabla `gift_cards`. Flujo de canje en checkout.	Fase 3
Multi-country	MercadoPago soporta MX, CO, AR. Agregar `currency` dinámico y opciones de envío por país.	Fase 3

Self-hosted inference	GPU pool (RunPod/Lambda). Reduce costo de pre-generación ~80%. Solo justificado a escala.	Fase 3
Account system	Supabase Auth para compradores (optional). Historial de pedidos, re-order.	Fase 2-3

13. Gotchas y Auto-Blindaje

Next.js 15 + App Router

- **Server Components son default**: Todo componente es RSC a menos que use `"use client"`. El configurador DEBE ser Client Component (usa estado, eventos).
- **Server Actions y forms**: Usar `useFormStatus` para loading states. Las Server Actions NO deben exponer datos sensibles en el return.
- **Image optimization**: `next/image` requiere configurar `remotePatterns` en `next.config.ts` para Supabase Storage URLs.
- **ISR y revalidación**: Páginas del catálogo pueden usar `revalidate = 3600` (1 hora). Revalidar on-demand cuando se publica un libro.

Supabase

- **RLS siempre activo**: TODAS las queries desde el client usan `anon key` y respetan RLS. Queries desde server con `service role` bypasean RLS (solo para composición y admin).
- **Storage policies**: Configurar policies en cada bucket. `illustrations` = público, `illustrations-hires` = solo service role.
- **Connection pooling**: Usar Supabase connection string con pooler para serverless (puerto 6543, no 5432).
- **Tipos generados**: Correr `supabase gen types typescript` después de cada migración.

fal.ai

- **URLs temporales**: Las URLs de imágenes generadas EXPIRAN. Descargar y almacenar en Supabase Storage inmediatamente después de generación.
- **512 token limit**: Prompts de FLUX Kontext tienen límite estricto. El prompt builder debe validar length.
- **Rate limits**: En capa gratuita, ~10 req/min. Inngest debe throttlear la concurrencia.

MercadoPago

- **Webhook idempotencia**: MercadoPago puede enviar el mismo webhook múltiples veces. Usar `payment_id` como clave de idempotencia.
- **Modo sandbox**: Usar credenciales de test durante desarrollo. Tarjetas de prueba disponibles en docs.
- **IPN vs Webhooks**: Usar el sistema de webhooks nuevo (v2), no el IPN legacy.

Inngest

- **Step timeouts**: Cada `step.run()` tiene timeout de 5 minutos. Si la generación de una imagen tarda más, dividir en: request → poll status.
- **Idempotencia de steps**: Cada step debe ser idempotente. Si se reintenta, no debe crear duplicados.
- **Event naming**: Usar formato `entity/action` (ej: `book/generate-all-variants`, `variant/generate-single`).

14. Convenciones de Código

Aspecto	Convención
Variables/Funciones	camelCase
Componentes React	PascalCase
Archivos/Carpetas	kebab-case
Constantes	UPPER_SNAKE_CASE
DB columns	snake_case
Commits	Conventional Commits (`feat:`, `fix:`, `chore:`)
Max file length	300 líneas (refactorizar si excede)
TypeScript `any`	NEVER — usar `unknown` y narrowing
Imports	Absolute paths con `@/` alias
Exports	Named exports (no default, excepto pages)
Error handling	try/catch con error codes tipados
Comments	Solo para lógica compleja. El código debe ser auto-explicativo.

Tech Spec generado con el pipeline de App Factory Pendiente aprobación antes de avanzar al siguiente skill

03

Image Generation Guidelines

v2.0 - Pipeline de pre-generacion, prompts y QA

TIPITI-IMAGE-GENERATION-GUIDELINES.md

> *Documento técnico de referencia para generación de imágenes con IA Versión: 2.0 | Febrero 2026 Uso: Este documento es la fuente de verdad para todos los documentos técnicos del proyecto (PDR, Tech Spec, User Stories, Blueprint). Cualquier decisión de generación de imágenes debe alinearse con estos lineamientos.*

1. Decisión Arquitectónica: Pre-Generación por Variantes

1.1 Modelo de Pre-Generación (v2.0)

Tipiti Books adopta un modelo de **pre-generación por variantes de personaje**. En lugar de generar imágenes bajo demanda por cada compra, se pre-generan TODAS las combinaciones posibles de personaje para cada libro del catálogo. Al momento de la compra, el sistema busca el set de ilustraciones pre-generadas que corresponde a la selección del comprador y le superpone los textos personalizados (nombre + dedicatoria) como capa de texto, no como parte de la imagen.

```
MODELO ANTERIOR (descartado):
 Compra ' Generar imágenes con IA ' QA ' Imprimir ' Enviar
 Costo por venta: $1-2 USD (generación) + riesgo de inconsistencia

MODELO ACTUAL (adoptado):
[Una vez] Admin genera + cura las 96 variantes por libro
 Compra ' Lookup variante ' Superponer texto (nombre) ' Imprimir ' Enviar
 Costo por venta: $0 USD en generación. Costo fijo total: ~$42 por libro.
```

1.2 Combinatoria de Variantes (V1)

Atributo	Opciones	Valores
Género	2	niño, niña
Color de pelo	4	rubio, castaño, pelirrojo, negro
Tipo de pelo	2	liso, ondulado
Tono de piel	3	claro, mate, oscuro
Lentes	2	sí, no
Ojos	1 (fijo)	puntitos negros
Total combinaciones	**96**	por libro, por idioma

Imágenes por variante: 1 portada + 9 spreads + 1 página final = **11 imágenes Total por libro:** 96 × 11 = **1,056 imágenes Costo total:** 1,056 × $0.04 = **~$42 USD** (una sola vez por libro)

1.3 Arquitectura de 3 Capas Independientes

Tipiti Books se estructura en **3 capas que se crean, escalan y versionan de forma totalmente independiente**. Este principio arquitectónico es la base de todas las decisiones de generación de imágenes, composición y almacenamiento.

```
CAPA 3 — TEXTO PERSONALIZADO (por compra)
Nombre del niño ({name}), dedicatoria, idioma
Se genera al instante. Costo: $0. Escala: infinita.
'NO es parte de la imagen. Se superpone como texto.
                              $
CAPA 2 — PERSONAJE (por variante, 96 combinaciones)
Ilustraciones pre-generadas con FLUX Kontext + curadas
Se genera UNA vez por libro. Costo: ~$42-120. Escala: fija.
'ES la imagen. Se almacena en character_variants/variant_pages
                              $
CAPA 1 — EL CUENTO (por libro del catálogo)
Escenas, ambientes, easter eggs, estructura narrativa
Se crea y cura UNA vez. Escala: manual (Sofi agrega libros).
'Define QUÉ se genera. Es el "template" (books + scenes).
```

Implicación clave para generación de imágenes: La Capa 2 (ilustraciones) NUNCA contiene elementos de la Capa 3 (texto, nombres). Esto es una regla inquebrantable del prompt builder: los prompts no incluyen el nombre del niño, y el NEGATIVE_BLOCK incluye `NO text, NO letters, NO words`. Todo texto se superpone después, en la fase de composición.

Implicación para escalabilidad:

- Agregar un idioma (V2) = solo agregar texto en Capa 3. Las 1,056 ilustraciones no se tocan.
- Corregir un texto = solo editar `scenes.text_narrative`. Las ilustraciones no se regeneran.
- Agregar un libro = crear escenas (Capa 1) + generar 96 variantes (Capa 2). La Capa 3 funciona automáticamente.

1.4 Composición del Archivo Final (Superposición de Capas)

Al momento de la compra, las 3 capas se combinan para producir el archivo de impresión:

```
Archivo final para impresión (por página) =
 Capa 1: Template de escena (scenes 'define composición, posición de texto)
 Capa 2: Ilustración pre-generada (variant_pages 'la imagen)
 Capa 3: Texto personalizado superpuesto:
- Texto narrativo con {name} reemplazado (scenes.text_narrative)
- Dedicatoria (solo en página designada)
- (V2: Texto en segundo idioma, usando espacio negativo reservado)
```

1.5 Justificación Técnica del Motor

Criterio	IP-Adapter/InstantID (descartado)	FLUX Kontext (adoptado)
Consistencia en múltiples poses	Se degrada en ángulos laterales	94% fidelidad, robusto en poses variadas

Edición iterativa	Cada regeneración introduce drift	Preserva identidad tras múltiples ediciones
Identidad artística vs fotográfica	Optimizado para fotos reales	Funciona nativamente con estilos artísticos
Complejidad operacional	ComfyUI + SDXL + LoRA + IP-Adapter	Una API call por imagen, zero DevOps
Time to market	Semanas-meses de setup	Días

2. Stack de Generación de Imágenes

2.1 Motor Principal: FLUX Kontext Pro

- **Modelo:** FLUX.1 Kontext [pro] — 12B parámetros, flow matching transformer
- **Provider:** fal.ai (managed inference)
- **Endpoint image-to-image:** `fal-ai/flux-pro/kontext`
- **Endpoint text-to-image:** `fal-ai/flux-pro/kontext/text-to-image`
- **Endpoint multi-referencia:** `fal-ai/flux-pro/kontext/max/multi` (hasta N imágenes de referencia)
- **Costo:** $0.04/imagen (pro), $0.08/imagen (max)
- **Autenticación:** API Key vía variable de entorno `FAL_KEY`
- **Contexto de uso:** Herramienta de administración para pre-generación. NO se llama en tiempo de compra.

2.2 Motor Auxiliar: FLUX + LoRA de Estilo (Fase 2+)

- **Trainer:** `fal-ai/flux-kontext-trainer` o `fal-ai/flux-lora-fast-training`
- **Inferencia con LoRA:** `fal-ai/flux-lora`
- **Costo de entrenamiento:** ~$2 por run (1000 steps)
- **Trigger word:** Token único por estilo (ej: `tipiti_watercolor_v1`)

2.3 Almacenamiento

- **Ilustraciones pre-generadas:** Supabase Storage (bucket `illustrations`)
- **Ilustraciones hi-res (print-ready):** Supabase Storage (bucket `illustrations-hires`)
- **Portraits de variante:** Supabase Storage (bucket `portraits`)
- **LoRAs entrenados:** fal.ai (URL devuelto post-entrenamiento) + referencia en Supabase

3. Modelo de Datos — Pre-Generación

3.1 Schema de la Tabla `character_variants`

Cada variante es una combinación única de rasgos físicos para un libro específico. No está vinculada a un usuario ni a un pedido — es un "template visual" pre-generado.

```
CREATE TABLE character_variants (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 book_id UUID REFERENCES books(id) NOT NULL,

-- Combinación de rasgos (define la unicidad)
 gender TEXT NOT NULL,                    -- 'boy', 'girl'
 hair_color TEXT NOT NULL,                -- 'blonde', 'brown', 'red', 'black'
 hair_type TEXT NOT NULL,                 -- 'straight', 'wavy'
 skin_tone TEXT NOT NULL,                 -- 'light', 'medium', 'dark'
 has_glasses BOOLEAN NOT NULL DEFAULT FALSE,

-- Rasgos fijos (no configurables por el comprador)
 eye_style TEXT DEFAULT 'black_dots',     -- siempre puntitos negros en V1

-- Imágenes de referencia generadas
 portrait_url TEXT,                       -- portrait frontal base
 character_sheet_urls JSONB,             -- {"front": "url", "three_quarter": "url", "side": "url", "happy":
"url", "sleepy": "url"}

-- Estado de curación
 status TEXT DEFAULT 'pending',           -- 'pending', 'generating', 'generated', 'approved', 'rejected'
 approved_by TEXT,                        -- admin que aprobó
 approved_at TIMESTAMPTZ,

-- Tracking
 created_at TIMESTAMPTZ DEFAULT now(),
 updated_at TIMESTAMPTZ DEFAULT now(),

-- Unicidad: una sola variante por combinación por libro
 UNIQUE(book_id, gender, hair_color, hair_type, skin_tone, has_glasses)
);

-- Índice para lookup rápido en tiempo de compra
CREATE INDEX idx_variant_lookup ON character_variants(book_id, gender, hair_color, hair_type, skin_tone,
has_glasses)
 WHERE status = 'approved';
```

3.2 Schema de la Tabla `variant_pages`

Cada variante tiene un set completo de páginas pre-generadas (1 por escena del libro).

```
CREATE TABLE variant_pages (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 variant_id UUID REFERENCES character_variants(id) NOT NULL,
 scene_id UUID REFERENCES scenes(id) NOT NULL,

-- Imagen pre-generada
 image_url TEXT NOT NULL,                 -- ilustración estándar (web preview)
 image_url_hires TEXT,                    -- versión upscaled para impresión (300 DPI)

-- Metadata de generación (para debugging y regeneración)
 prompt_used TEXT NOT NULL,
 seed_used INTEGER,
 generation_model TEXT NOT NULL,          -- 'flux-kontext-pro', 'flux-kontext-max', etc.
```

```
 generation_cost DECIMAL(6,4),

-- Estado de curación
 status TEXT DEFAULT 'generated',         -- 'generating', 'generated', 'approved', 'rejected'
 retry_count INTEGER DEFAULT 0,

 created_at TIMESTAMPTZ DEFAULT now(),

 UNIQUE(variant_id, scene_id)
);
```

3.3 Tabla `books` (actualizada)

```
CREATE TABLE books (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 slug TEXT UNIQUE NOT NULL,                -- 'buenas-noches'
 title_template TEXT NOT NULL,             -- 'Buenas Noches, {name}'
 style_prompt TEXT NOT NULL,               -- STYLE_BLOCK fijo del libro
 style_lora_url TEXT,                      -- URL del LoRA de estilo (Fase 2+)
 total_scenes INTEGER NOT NULL,            -- 9 (spreads) + portada + final = 11
 target_age TEXT DEFAULT '0-3',            -- rango de edad objetivo
 default_language TEXT DEFAULT 'es',
 available_languages TEXT[] DEFAULT '{"es"}',

-- Estado del catálogo
 total_variants INTEGER DEFAULT 96,        -- cuántas variantes tiene este libro
 approved_variants INTEGER DEFAULT 0,      -- cuántas están aprobadas
 is_published BOOLEAN DEFAULT FALSE,       -- visible en el catálogo público

-- Impresión
 page_width_cm DECIMAL(4,1) DEFAULT 22.0,
 page_height_cm DECIMAL(4,1) DEFAULT 18.0,
 total_pages INTEGER DEFAULT 20,           -- incluyendo portada y final

 cover_template_url TEXT,
 created_at TIMESTAMPTZ DEFAULT now()
);
```

3.4 Tabla `scenes` (actualizada)

```
CREATE TABLE scenes (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 book_id UUID REFERENCES books(id) NOT NULL,
 scene_number INTEGER NOT NULL,            -- 0=portada, 1-9=spreads, 10=final

-- Descripción visual para el prompt de IA
 visual_description TEXT NOT NULL,
 camera_angle TEXT DEFAULT 'medium_shot',
 lighting TEXT DEFAULT 'warm',
 emotion TEXT DEFAULT 'peaceful',
 suggested_seed INTEGER,

-- Elementos recurrentes (easter eggs)
 recurring_elements TEXT[],                -- ['sleeping_mouse', 'teddy_bear', 'butterfly']

-- Composición de la página
 character_position TEXT DEFAULT 'left',   -- 'left', 'right' (alterna, NUNCA center)
 text_position TEXT DEFAULT 'top_left',    -- posición del bloque de texto (alterna para dinamismo)
```

```
 text_position_secondary TEXT,             -- posición del segundo idioma (V2)

-- Textos narrativos (con placeholder {name})
 text_narrative JSONB NOT NULL,            -- {"es": "Buenas noches, {name}. Shhh...", "de": "Gute Nacht,
{name}. Shhh..."}

 UNIQUE(book_id, scene_number)
);
```

3.5 Tabla `orders` (simplificada)

```
CREATE TABLE orders (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 variant_id UUID REFERENCES character_variants(id) NOT NULL,
 book_id UUID REFERENCES books(id) NOT NULL,

-- Personalización del comprador
 child_name TEXT NOT NULL,                 -- se inyecta en {name} de los textos
 dedication TEXT,                          -- dedicatoria personal
 language TEXT DEFAULT 'es',               -- idioma elegido para los textos
 second_language TEXT,                     -- segundo idioma (V2, nullable)

-- Comprador
 buyer_email TEXT NOT NULL,
 buyer_name TEXT NOT NULL,
 buyer_phone TEXT,

-- Envío
 shipping_address JSONB NOT NULL,
 shipping_city TEXT,
 shipping_region TEXT,
 shipping_country TEXT DEFAULT 'CL',

-- Pago
 payment_provider TEXT DEFAULT 'mercadopago',
 payment_id TEXT,                          -- ID de transacción de MercadoPago
 payment_status TEXT DEFAULT 'pending',    -- 'pending', 'approved', 'rejected'
 amount_paid DECIMAL(10,2),
 currency TEXT DEFAULT 'CLP',

-- Fulfillment
 print_file_url TEXT,                      -- URL del PDF listo para imprenta
 print_status TEXT DEFAULT 'pending',      -- 'pending', 'sent_to_print', 'printed', 'received_by_admin'
 shipping_status TEXT DEFAULT 'pending',   -- 'pending', 'shipped', 'delivered'
 tracking_number TEXT,

-- Estado general
 status TEXT DEFAULT 'created',            -- 'created', 'paid', 'composing', 'printing', 'shipped',
'delivered', 'cancelled'

 created_at TIMESTAMPTZ DEFAULT now(),
 updated_at TIMESTAMPTZ DEFAULT now()
);
```

3.6 Mapeo de Campos del Comprador a Variante

El comprador selecciona opciones en lenguaje natural. El sistema las mapea a los valores de la tabla `character_variants` para hacer el lookup.

```
// Mapeo de opciones del UI a valores de character_variants
const BUYER_TO_VARIANT_MAP = {
 gender: {
'Niño': 'boy',
'Niña': 'girl',
},
 hair_color: {
'Rubio': 'blonde',
'Castaño': 'brown',
'Pelirrojo': 'red',
'Negro': 'black',
},
 hair_type: {
'Liso': 'straight',
'Ondulado': 'wavy',
},
 skin_tone: {
'Clara': 'light',
'Mate': 'medium',
'Oscura': 'dark',
},
 has_glasses: {
'Sí': true,
'No': false,
},
};

// Lookup de variante en tiempo de compra
async function findVariant(
 bookId: string,
 selection: BuyerSelection
): Promise<CharacterVariant | null> {
 const { data } = await supabase
.from('character_variants')
.select('*, variant_pages(*)')
.eq('book_id', bookId)
.eq('gender', BUYER_TO_VARIANT_MAP.gender[selection.gender])
.eq('hair_color', BUYER_TO_VARIANT_MAP.hair_color[selection.hairColor])
.eq('hair_type', BUYER_TO_VARIANT_MAP.hair_type[selection.hairType])
.eq('skin_tone', BUYER_TO_VARIANT_MAP.skin_tone[selection.skinTone])
.eq('has_glasses', BUYER_TO_VARIANT_MAP.has_glasses[selection.glasses])
.eq('status', 'approved')
.single();

 return data;
}
```

4. Prompt Builder — Especificación Técnica

4.1 Estructura del Prompt

Cada prompt de generación se compone de 4 bloques concatenados:

```
[STYLE_BLOCK] + [CHARACTER_BLOCK] + [SCENE_BLOCK] + [NEGATIVE_BLOCK]
```

Límite: 512 tokens máximo (restricción de FLUX Kontext).

4.2 STYLE_BLOCK — Estilo Artístico (Fijo por Libro)

```
soft watercolor children's book illustration, gentle visible brush strokes,
warm pastel color palette, hand-painted storybook art, whimsical and tender,
editorial quality children's picture book, textured watercolor paper background
```

4.3 CHARACTER_BLOCK — Identidad del Personaje (Generado por Variante)

Se construye a partir de los campos de `character_variants`. Ya no incluye `{name}` porque el nombre no aparece en la imagen.

Reglas de construcción:

- **Empezar con edad y género:** `"a toddler [boy/girl]"`
- **Listar TODOS los rasgos físicos:** piel → pelo (estilo + color) → ojos (puntitos negros)
- **Accesorios con énfasis:** `"who ALWAYS wears small round glasses"`
- **Repetir accesorios críticos al final** (anti-drift)
- **NO incluir nombre** — el nombre es texto superpuesto, no parte de la imagen

Ejemplo:

```
a toddler boy with warm olive skin, short dark brown wavy hair,
small black dot eyes, who ALWAYS wears small round glasses,
wearing comfortable pajamas,
the child ALWAYS has small round glasses visible on the face
```

```
// Mapeo de variante a descriptores
const SKIN_TONE_PROMPT: Record<string, string> = {
'light': 'very light/pale skin',
'medium': 'warm olive/medium skin',
'dark': 'warm deep brown skin',
};

const HAIR_COLOR_PROMPT: Record<string, string> = {
'blonde': 'golden blonde',
'brown': 'dark brown',
'red': 'red',
'black': 'jet black',
};

const HAIR_TYPE_PROMPT: Record<string, string> = {
'straight': 'straight',
'wavy': 'wavy',
};

const GENDER_HAIR_STYLE: Record<string, string> = {
'boy_straight': 'short straight hair',
'boy_wavy': 'short wavy hair',
'girl_straight': 'long straight hair',
'girl_wavy': 'long wavy hair',
};
```

```
function buildCharacterBlock(variant: CharacterVariant): string {
 const gender = variant.gender; // 'boy' or 'girl'
 const hairKey = `${gender}_${variant.hair_type}`;

 const parts: string[] = [
`a toddler ${gender}`,
`with ${SKIN_TONE_PROMPT[variant.skin_tone]}`,
`${HAIR_COLOR_PROMPT[variant.hair_color]} ${GENDER_HAIR_STYLE[hairKey]}`,
'small black dot eyes',  // SIEMPRE puntitos negros
];

 if (variant.has_glasses) {
 parts.push('who ALWAYS wears small round glasses');
}

 parts.push('wearing comfortable pajamas');

// Repetición anti-drift
 if (variant.has_glasses) {
 parts.push('the child ALWAYS has small round glasses visible on the face');
}

 return parts.join(', ');
}
```

4.4 SCENE_BLOCK — Descripción de Escena

Se almacena en `scenes.visual_description`. Incluye composición y elementos recurrentes.

Estructura:

```
[acción del personaje] + [ubicación/fondo] + [iluminación] + [emoción] +
[posición del personaje: left/right] + [elementos recurrentes/easter eggs] +
[espacio negativo para texto: posición específica]
```

Ejemplo:

```
sitting on a cozy bed surrounded by soft pillows, in a warmly lit bedroom
with stars visible through the window, nighttime, peaceful sleepy expression,
gentle warm lamplight, the child positioned on the LEFT side of the image,
a small sleeping mouse curled up on the windowsill, a teddy bear peeking
from behind a pillow, clear negative space on the upper right for text overlay
```

Reglas de composición:

- **Personaje NUNCA en el centro** (zona del lomo/pliegue del libro)
- **Alternar posición:** escenas impares = izquierda, pares = derecha
- **Espacio negativo dinámico para texto:** alterna posición (top-left, bottom-left, top-right, bottom-right)
- **Espacio negativo adicional reservado** para segundo idioma en V2
- **Incluir elementos recurrentes** (easter eggs): un mismo animalito/peluche que aparece en cada escena en un lugar diferente

4.5 NEGATIVE_BLOCK — Anti-Slop

```
NOT 3D render, NOT CGI, NOT Pixar style, NOT Disney 3D, NOT photorealistic,
NOT plastic, NOT shiny, NOT glossy, NOT digital art, NOT anime, NOT manga,
NOT cartoon network style, NOT clipart, NOT stock illustration, NO text,
NO letters, NO words, NO writing in the image
```

Nota v2.0: Se agrega `NO text, NO letters, NO words, NO writing in the image` porque todo el texto se superpone como capa separada. La imagen debe estar 100% libre de texto generado por IA.

4.6 Prompt Completo

```
function buildFullPrompt(variant: CharacterVariant, scene: Scene, book: Book): string {
 const style = book.style_prompt;
 const character = buildCharacterBlock(variant);
 const sceneDesc = scene.visual_description;
 const negative = NEGATIVE_PROMPT;

 const prompt = `${style}, ${character}, ${sceneDesc}. ${negative}`;

 if (prompt.split(' ').length > 350) {
 console.warn('Prompt approaching 512 token limit');
}

 return prompt;
}
```

5. Pipeline de Pre-Generación — Flujo Técnico

5.1 Flujo General (Herramienta de Admin)

```
Panel Admin (Sofi):
1. Selecciona un libro del catálogo
2. Clickea "Generar todas las variantes"
3. Sistema genera las 96 combinaciones en batch (background job)
 Para cada variante:
 a. Genera portrait base (text-to-image)
 b. Genera character sheet (4 vistas, image-to-image)
 c. Genera 11 páginas (portada + 9 spreads + final)
 d. Upscalea a resolución de impresión
4. Sofi revisa cada variante en el panel:
- Aprueba variantes correctas
- Rechaza/regenera las que no le gustan
- Puede editar la descripción visual de una escena y regenerar solo esa página
5. Cuando TODAS las variantes están aprobadas, publica el libro al catálogo
```

5.2 Generación del Portrait por Variante

```
// Para cada una de las 96 variantes
async function generatePortrait(variant: CharacterVariant, book: Book) {
 const prompt = `${book.style_prompt}, ${buildCharacterBlock(variant)}, front view, neutral pose, looking
at viewer, centered portrait, simple soft background. ${NEGATIVE_PROMPT}`;

 const result = await fal.subscribe('fal-ai/flux-pro/kontext/text-to-image', {
 input: {
 prompt,
 aspect_ratio: '1:1',
 num_images: 3,           // 3 opciones para elegir
 guidance_scale: 3.5,
 output_format: 'png',
},
});

// Sofi elige la mejor ' se guarda como portrait_url
 return result.images;
}
```

5.3 Generación de Páginas por Variante

```
async function generateVariantPages(variant: CharacterVariant, book: Book, scenes: Scene[]) {
 for (const scene of scenes) {
 const prompt = buildFullPrompt(variant, scene, book);

 const result = await fal.subscribe('fal-ai/flux-pro/kontext', {
 input: {
 prompt,
 image_url: variant.portrait_url,  // referencia de identidad
 guidance_scale: 3.5,
 num_images: 2,                     // 2 opciones por escena
 output_format: 'png',
},
});

// Guardar en variant_pages con status 'generated'
 await saveVariantPage(variant.id, scene.id, result.images[0].url, prompt);
}
}
```

5.4 Batch Generation (las 96 variantes)

```
async function generateAllVariants(bookId: string) {
 const book = await getBook(bookId);
 const scenes = await getScenes(bookId);

 const allCombinations = generateCombinations(); // 96 combinaciones

 for (const combo of allCombinations) {
// Crear registro de variante
 const variant = await createVariant(bookId, combo);

// Generar portrait (Sofi aprueba después)
 await generatePortrait(variant, book);
```

```
// Generar todas las páginas
 await generateVariantPages(variant, book, scenes);

// Marcar como 'generated' (pendiente aprobación)
 await updateVariantStatus(variant.id, 'generated');
}
}

function generateCombinations(): VariantCombo[] {
 const genders = ['boy', 'girl'];
 const hairColors = ['blonde', 'brown', 'red', 'black'];
 const hairTypes = ['straight', 'wavy'];
 const skinTones = ['light', 'medium', 'dark'];
 const glasses = [true, false];

 const combos: VariantCombo[] = [];
 for (const g of genders)
 for (const hc of hairColors)
 for (const ht of hairTypes)
 for (const st of skinTones)
 for (const gl of glasses)
 combos.push({ gender: g, hair_color: hc, hair_type: ht, skin_tone: st, has_glasses: gl });

 return combos; // 96 combinaciones
}
```

5.5 Costo de Pre-Generación por Libro

Paso	Imágenes	Costo unitario	Total
Portraits (3 opciones × 96 variantes)	288	$0.04	$11.52
Character sheets (4 vistas × 96)	384	$0.04	$15.36
Páginas (11 × 96, 2 opciones cada una)	2,112	$0.04	$84.48
Regeneraciones (~10% retry)	~211	$0.04	$8.44
Total por libro	**~2,995**	**~$120 USD**	

Nota: Este es el costo máximo si se generan 2 opciones por página. Si se genera 1 sola por página y se regenera solo las rechazadas, el costo baja a **~$60 USD** por libro.

Costo amortizado: A 50 ventas del primer mes, el costo de generación por libro vendido es **$1.20-2.40 USD**. A 200 ventas, baja a **$0.30-0.60 USD**.

6. Pipeline de Compra — Composición en Tiempo Real

6.1 Flujo de Composición (al momento de la compra)

Cuando un comprador finaliza su personalización y paga:

```
1. LOOKUP: Buscar character_variant por (book_id, gender, hair_color, hair_type, skin_tone, has_glasses)
2. FETCH: Obtener todas las variant_pages (11 imágenes pre-generadas)
3. COMPOSE: Para cada página:
 a. Cargar ilustración pre-generada (image_url_hires)
 b. Reemplazar {name} en text_narrative con child_name
 c. Renderizar texto sobre la imagen en la posición definida (text_position)
 d. Aplicar tipografía del libro (font_family, font_size, font_color)
4. ASSEMBLE: Generar PDF listo para imprenta (20 páginas, 22×18 cm, 300 DPI)
5. STORE: Guardar PDF en Supabase Storage ' actualizar order.print_file_url
6. NOTIFY: Email de confirmación al comprador + notificación a Sofi
```

6.2 Composición de Texto sobre Imagen

```
// Pseudocódigo de composición
interface PageComposition {
 illustration_url: string;     // imagen pre-generada
 text: string;                 // texto con {name} reemplazado
 text_position: TextPosition;  // 'top_left', 'bottom_left', 'top_right', 'bottom_right'
 font: FontConfig;             // tipografía del libro
}

function composePageText(
 scene: Scene,
 childName: string,
 language: string
): string {
 const template = scene.text_narrative[language]; // "Buenas noches, {name}. Shhh..."
 return template.replace(/{name}/g, childName);   // "Buenas noches, Alonso. Shhh..."
}
```

7. Reglas de Prompting para Consistencia — Cheat Sheet

7.1 HACER (DO)

Regla	Ejemplo
Referir al personaje con descriptores específicos	"The toddler boy with olive skin and wavy brown hair"
Repetir accesorios críticos 2x en el prompt	Al inicio Y al final del CHARACTER_BLOCK
Especificar posición del personaje (left/right)	"the child positioned on the LEFT side of the image"
Incluir espacio negativo explícito para texto	"clear negative space on the upper right for text overlay"
Especificar "NO text, NO letters" en el negative	Evita que la IA genere texto dentro de la imagen
Incluir elementos recurrentes (easter eggs)	"a small sleeping mouse on the windowsill"

Usar el mismo camera language entre spreads	Consistencia de ángulos y encuadres

7.2 NO HACER (DON'T)

Anti-patrón	Por qué falla
Poner al personaje en el centro de la imagen	Zona del pliegue del libro — se pierde al encuadernar
Incluir el nombre del niño en el prompt de imagen	El nombre es texto superpuesto, no generado por IA
Omitir accesorios del prompt	El modelo los elimina si no se mencionan
Generar texto con IA dentro de la imagen	La IA genera texto ilegible/incorrecto; usar overlay
Exceder ~350 palabras en el prompt	Límite de 512 tokens; trunca silenciosamente

8. Quality Assurance — Validación de Variantes

8.1 QA por Variante (una sola vez)

Cada variante se valida UNA vez durante la pre-generación. Una vez aprobada, se sirve idéntica a todos los compradores con esa combinación.

Checklist por variante:

- **Portrait:** ¿Se corresponde con la combinación de rasgos? ¿Estilo acuarela?
- **Consistencia entre páginas:** ¿El personaje se ve igual en las 11 páginas?
- **Accesorios:** ¿Lentes presentes en TODAS las páginas? (si aplica)
- **Composición:** ¿Personaje a izquierda/derecha (nunca centro)? ¿Espacio negativo para texto?
- **Elementos recurrentes:** ¿Los easter eggs están presentes y consistentes?
- **Sin texto en imagen:** ¿La imagen está libre de letras/palabras generadas?
- **Sin artefactos:** ¿Manos correctas? ¿Sin distorsiones?

8.2 Estrategia de Regeneración

Si una página de una variante no pasa QA, Sofi puede regenerar solo esa página:

- **Retry 1:** Mismo prompt, diferente seed
- **Retry 2:** Aumentar guidance_scale a 4.0-4.5
- **Retry 3:** Usar Kontext Max con múltiples vistas del character sheet
- **Retry 4:** Editar la descripción visual de la escena y regenerar

9. Evolución por Fases

Fase 1 — MVP (Meses 1-3)

```
Pre-generación: 96 variantes × 1 libro = ~$60-120 USD (una vez)
Compra: lookup + composición texto + PDF ' $0 en generación por venta
Validación: manual por Sofi (una vez por variante)
Estilo: solo vía prompt engineering (STYLE_BLOCK)
```

Fase 2 — Calidad + Expansión (Meses 4-8)

```
Pre-generación con Style LoRA ' estilo más consistente
Segundo libro en catálogo (nuevas 96 variantes)
Opción bilingüe (texto en segundo idioma usando espacio negativo)
QA semi-automatizado (CLIP scoring para pre-filtrar)
Segundo personaje ' generación on-demand solo para el 2do personaje
```

Fase 3 — Escala (Mes 9+)

```
Self-hosted inference para bajar costos de pre-generación
Más libros en catálogo
V2 features: foto del regalador, bundle con peluche
Auto-generación de nuevas variantes si se agregan opciones de personalización
```

10. Formatos de Imagen y Resolución

Uso	Resolución	Aspect Ratio	Formato
Portrait de variante	1024×1024	1:1	PNG
Character sheet (cada vista)	1024×1024	1:1	PNG
Spread doble (22×18 cm × 2)	2048×1024	~2:1	PNG
Spread hi-res (impresión 300 DPI)	5197×2362	~2.2:1	PNG
Portada	1024×1536	~2:3	PNG
Preview web (thumbnail)	512×256	2:1	JPEG (q85)

Nota: Las imágenes de 22×18 cm a 300 DPI requieren 2598×2126 px por página, o 5197×2126 px por spread doble. Las imágenes generadas por FLUX (~2048px) se upscalean con Real-ESRGAN como paso post-generación durante la pre-generación.

11. Parámetros API de Referencia Rápida

FLUX Kontext Pro — Text-to-Image (portraits)

```
const result = await fal.subscribe('fal-ai/flux-pro/kontext/text-to-image', {
 input: {
 prompt: `${STYLE_BLOCK}, ${buildCharacterBlock(variant)}, front view, neutral pose, looking at viewer,
centered portrait, simple soft background. ${NEGATIVE_PROMPT}`,
 aspect_ratio: '1:1',
 num_images: 3,
 guidance_scale: 3.5,
 output_format: 'png',
},
});
```

FLUX Kontext Pro — Image-to-Image (escenas)

```
const result = await fal.subscribe('fal-ai/flux-pro/kontext', {
 input: {
 prompt: buildFullPrompt(variant, scene, book),
 image_url: variant.portrait_url,
 guidance_scale: 3.5,
 num_images: 2,
 output_format: 'png',
},
});
```

FLUX Kontext Max — Multi-Referencia (retry de alta calidad)

```
const result = await fal.subscribe('fal-ai/flux-pro/kontext/max/multi', {
 input: {
 prompt: buildFullPrompt(variant, scene, book),
 image_urls: [
 variant.portrait_url,
 variant.character_sheet_urls.three_quarter,
 variant.character_sheet_urls.side,
],
 guidance_scale: 3.5,
 num_images: 2,
 output_format: 'png',
},
});
```

12. Fuentes y Referencias Técnicas

- FLUX.1 Kontext Pro API — fal.ai

- FLUX.1 Kontext Text-to-Image API — fal.ai
- FLUX.1 Kontext Max Multi-Reference — fal.ai
- FLUX Kontext Trainer — fal.ai
- BFL Prompting Guide — Image-to-Image
- BFL FLUX Kontext Announcement
- FLUX.1 Kontext Paper (arXiv)
- PricePerToken — Image Model Pricing

Este documento es la fuente de verdad técnica para generación de imágenes en Tipiti Books. v2.0 — Arquitectura de pre-generación por variantes. Todos los documentos del pipeline (PDR, Tech Spec, User Stories, Blueprint) deben referenciar estos lineamientos.